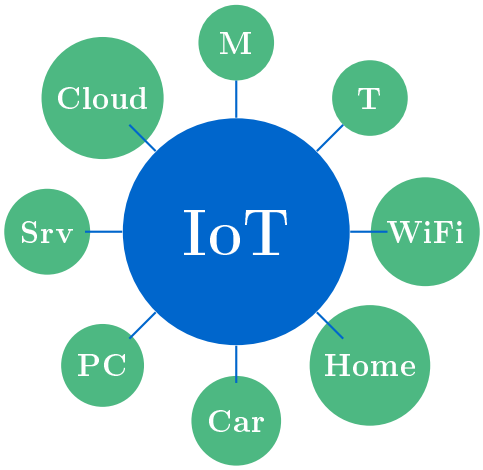


Internet des Objets

Fondements, Protocoles et Applications

Dr. BADR EL KHALYLY

Fruit de sa thèse doctorale et de son étude bibliographique approfondie

Ce cours synthétise les recherches doctorales menées sur l'Internet des Objets, incluant une revue exhaustive de la littérature scientifique du domaine.

Année Universitaire 2024-2025

Table des matières

1 Introduction à l'Internet des Objets **4**
- 1.1 Contexte et Motivation . . . 4
  - 1.1.1 Définition Formelle . . . 4
  - 1.1.2 Évolution Historique . . . 4
  - 1.1.3 Chiffres Clés et Perspectives . . . 5
- 1.2 Architecture d'un Objet Connecté . . . 5
  - 1.2.1 Définition et Composants . . . 5
  - 1.2.2 Caractéristiques Techniques . . . 5
- 1.3 Les Microcontrôleurs pour l'IoT . . . 6
  - 1.3.1 Définition et Rôle . . . 6
  - 1.3.2 Architecture Interne d'un Microcontrôleur . . . 6
  - 1.3.3 Plateformes de Développement IoT . . . 7
  - 1.3.4 Tableau Comparatif des Plateformes . . . 8
- 1.4 Les Capteurs . . . 9
  - 1.4.1 Principe de Fonctionnement . . . 9
  - 1.4.2 Chaîne d'Acquisition . . . 9
  - 1.4.3 Classification des Capteurs IoT . . . 10
  - 1.4.4 Exemple : Capteur de Température DS18B20 . . . 10
- 1.5 Les Actionneurs . . . 11
  - 1.5.1 Définition et Classification . . . 11
  - 1.5.2 Commande PWM . . . 11
- 1.6 Domaines d'Application de l'IoT . . . 12
  - 1.6.1 Smart City (Ville Intelligente) . . . 13
  - 1.6.2 Industrie 4.0 . . . 13
  - 1.6.3 Santé Connectée (e-Health) . . . 13
  - 1.6.4 Agriculture de Précision . . . 13

2 Pile de Protocoles de Communication IoT **15**
- 2.1 Introduction . . . 15
  - 2.1.1 Modèle de Référence IoT . . . 15
- 2.2 Couche Physique . . . 15
  - 2.2.1 Technologies Sans Fil à Courte Portée . . . 16
  - 2.2.2 Technologies LPWAN (Low Power Wide Area Network) . . . 17
  - 2.2.3 Comparaison Globale des Technologies . . . 19
- 2.3 Couche Réseau . . . 19
  - 2.3.1 IPv6 et 6LoWPAN . . . 19
  - 2.3.2 Protocole RPL . . . 20
- 2.4 Couche Transport . . . 20

2.4.1 TCP vs UDP pour l'IoT . . . 20
2.5 Couche Application . . . 20
2.5.1 MQTT (Message Queuing Telemetry Transport) . . . 20
2.5.2 CoAP (Constrained Application Protocol) . . . 22
2.5.3 Tableau Comparatif des Protocoles Applicatifs . . . 22

3 Niveaux de Déploiement : Edge, Fog et Cloud 23
3.1 Introduction . . . 23
3.2 Edge Computing . . . 24
3.2.1 Définition . . . 24
3.2.2 Avantages . . . 24
3.2.3 Technologies Edge . . . 24
3.3 Fog Computing . . . 24
3.3.1 Définition . . . 24
3.3.2 Fonctions du Fog . . . 24
3.4 Cloud Computing pour l'IoT . . . 25
3.4.1 Services Cloud . . . 25
3.4.2 Comparaison des Trois Niveaux . . . 25

4 Sécurité de l'Internet des Objets 26
4.1 Introduction et Enjeux . . . 26
4.1.1 Menaces Principales . . . 26
4.2 Sécurité par Couche . . . 26
4.2.1 Couche Physique . . . 26
4.2.2 Couche Communication . . . 27
4.2.3 Authentification et Autorisation . . . 27
4.3 Bonnes Pratiques de Sécurité IoT . . . 27

5 Projets Pratiques IoT 28
5.1 Projet 1 : Station Météo Connectée . . . 28
5.1.1 Objectifs Pédagogiques . . . 28
5.1.2 Matériel Requis . . . 28
5.1.3 Schéma de Câblage . . . 28
5.1.4 Code Source . . . 29
5.2 Projet 2 : Système de Contrôle d'Accès RFID . . . 31
5.2.1 Objectifs . . . 31
5.2.2 Matériel . . . 31
5.3 Projet 3 : Système d'Irrigation Intelligent . . . 31
5.3.1 Architecture du Système . . . 31

6 Intelligence Artificielle et Machine Learning pour l'IoT 32
6.1 Introduction à l'IA pour l'IoT . . . 32
6.1.1 Convergence IoT et IA . . . 32
6.1.2 Types d'Apprentissage Automatique . . . 33
6.2 TinyML : Machine Learning Embarqué . . . 33
6.2.1 Définition et Enjeux . . . 33
6.2.2 Frameworks TinyML . . . 33
6.2.3 Pipeline de Développement TinyML . . . 33
6.2.4 Techniques d'Optimisation des Modèles . . . 34

6.2.5 Exemple : Classification Audio sur Arduino . . . 34
6.3 Détection d'Anomalies dans les Données IoT . . . 36
6.3.1 Approches de Détection . . . 36
6.3.2 Algorithmes de Détection d'Anomalies . . . 37
6.3.3 Exemple : Autoencoder pour Détection d'Anomalies . . . 37
6.4 Maintenance Prédictive . . . 38
6.4.1 Principe . . . 38
6.4.2 Indicateurs de Santé des Équipements . . . 39
6.4.3 Modèle de Prédiction de RUL . . . 39
6.5 Federated Learning pour l'IoT . . . 40
6.5.1 Principe . . . 40
6.5.2 Avantages pour l'IoT . . . 40
6.5.3 Algorithme FedAvg . . . 41

7 Industrie 4.0 et Internet Industriel des Objets (IIoT) 42
7.1 Introduction à l'Industrie 4.0 . . . 42
7.1.1 Définition . . . 42
7.1.2 Les Neuf Piliers Technologiques . . . 42
7.2 Internet Industriel des Objets (IIoT) . . . 43
7.2.1 Différences entre IoT et IIoT . . . 43
7.2.2 Architecture de Référence IIoT . . . 43
7.3 Protocoles Industriels . . . 43
7.3.1 OPC UA (Unified Architecture) . . . 43
7.3.2 Modbus et ses Évolutions . . . 44
7.3.3 Protocoles Temps Réel . . . 44
7.4 Jumeaux Numériques (Digital Twins) . . . 44
7.4.1 Concept . . . 44
7.4.2 Applications des Jumeaux Numériques . . . 45
7.5 Cas d'Usage IIoT . . . 45
7.5.1 Monitoring de Ligne de Production . . . 45

8 Ville Intelligente (Smart City) 48
8.1 Concept et Enjeux . . . 48
8.1.1 Définition . . . 48
8.1.2 Les Six Dimensions de la Smart City . . . 48
8.2 Infrastructure IoT Urbaine . . . 49
8.2.1 Architecture Globale . . . 49
8.3 Gestion Intelligente du Trafic . . . 49
8.3.1 Composants du Système . . . 49
8.3.2 Algorithmes d'Optimisation . . . 49
8.4 Éclairage Public Intelligent . . . 52
8.4.1 Architecture du Système . . . 52
8.4.2 Logique de Contrôle . . . 52
8.5 Gestion Intelligente des Déchets . . . 55
8.5.1 Système de Poubelles Connectées . . . 55

9 Santé Connectée et Internet des Objets Médicaux (IoMT) 56
9.1 Introduction à l'IoT Médical . . . 56
9.1.1 Définition et Enjeux . . . 56
9.1.2 Marché et Perspectives . . . 57
9.2 Dispositifs Médicaux Connectés . . . 57
9.2.1 Classification des Dispositifs IoMT . . . 57
9.2.2 Exemple : Système de Monitoring Cardiaque . . . 57
9.3 Télémédecine et Monitoring à Distance . . . 60
9.3.1 Architecture de Télésurveillance . . . 60
9.3.2 Cas d'Usage : Suivi du Diabète . . . 60
9.4 Réglementation et Sécurité des Données de Santé . . . 61
9.4.1 Cadre Réglementaire . . . 61
9.4.2 Sécurité Spécifique IoMT . . . 61

10 Agriculture Intelligente (Smart Farming) 62
10.1 Introduction à l'Agriculture de Précision . . . 62
10.1.1 Définition . . . 62
10.2 Capteurs Agricoles . . . 63
10.2.1 Capteurs de Sol . . . 63
10.2.2 Système de Monitoring des Cultures . . . 63
10.3 Irrigation Intelligente . . . 65
10.3.1 Algorithme de Décision d'Irrigation . . . 65
10.4 Drones et Imagerie Agricole . . . 69
10.4.1 Applications des Drones . . . 69
10.4.2 Indice de Végétation NDVI . . . 69

11 Plateformes IoT et Services Cloud 70
11.1 Introduction aux Plateformes IoT . . . 70
11.2 Plateformes Cloud Majeures . . . 70
11.2.1 AWS IoT . . . 70
11.2.2 Azure IoT . . . 71
11.2.3 Google Cloud IoT . . . 71
11.2.4 Comparaison des Plateformes Cloud . . . 71
11.3 Plateformes Open Source . . . 72
11.3.1 ThingsBoard . . . 72
11.3.2 Eclipse IoT . . . 72
11.3.3 Exemple : Déploiement ThingsBoard . . . 72
11.4 Intégration et APIs . . . 73
11.4.1 Patterns d'Intégration . . . 73
11.4.2 Exemple d'API REST IoT . . . 73

Conclusion 77

A Glossaire 78

B Références des Composants 80
B.1 Microcontrôleurs . . . 80
B.2 Capteurs Courants . . . 80

Bibliographie **81**

Chapitre 1

Introduction à l'Internet des Objets

1.1 Contexte et Motivation

L'Internet des Objets (IoT - *Internet of Things*) représente une révolution technologique majeure qui étend le paradigme de l'Internet traditionnel aux objets physiques du quotidien. Cette évolution permet à des milliards d'appareils de collecter, échanger et traiter des données de manière autonome, créant ainsi un écosystème numérique interconnecté sans précédent.

1.1.1 Définition Formelle

Selon l'Union Internationale des Télécommunications (UIT-T Y.2060, 2012), l'Internet des Objets est défini comme :

> *"Une infrastructure mondiale pour la société de l'information, qui permet de disposer de services évolués en interconnectant des objets (physiques ou virtuels) grâce aux technologies de l'information et de la communication interopérables existantes ou en évolution."*

D'un point de vue technique, l'IoT peut être caractérisé par l'équation conceptuelle suivante :

$$\text{IoT} = \text{Objets Physiques} + \text{Capteurs/Actionneurs} + \text{Connectivité} + \text{Intelligence} \quad (1.1)$$

1.1.2 Évolution Historique

Le concept d'IoT a émergé progressivement au fil des décennies :

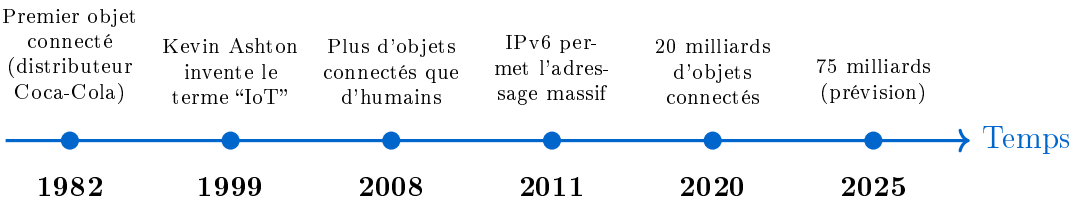


FIGURE 1.1 – Évolution historique de l'Internet des Objets

1.1.3 Chiffres Clés et Perspectives

L'IoT connaît une croissance exponentielle, comme l'illustrent les données suivantes :

TABLE 1.1 – Évolution du marché IoT mondial (Source : Statista, Gartner)

Indicateur	2020	2023	2025	2030
Objets connectés (milliards)	20,4	35,8	75,4	125+
Marché mondial (milliards $)	389	650	1100	2500+
Données générées (Zo/an)	18	73	175	600+

1.2 Architecture d'un Objet Connecté

1.2.1 Définition et Composants

Un **objet connecté** est un dispositif physique doté de capacités de perception, de traitement, de communication et d'action. Son architecture générique comprend les composants suivants :

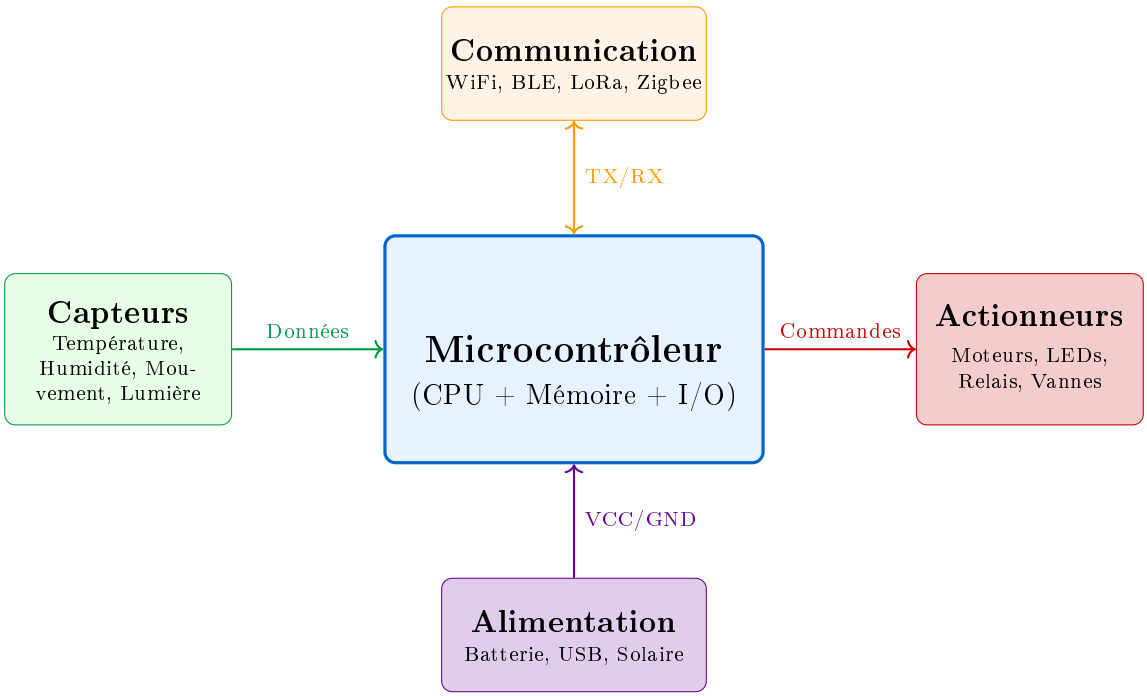


FIGURE 1.2 – Architecture générique d'un objet connecté

1.2.2 Caractéristiques Techniques

Les objets connectés présentent des contraintes spécifiques qui les distinguent des systèmes informatiques traditionnels :

TABLE 1.2 – Contraintes caractéristiques des objets IoT

Contrainte	Valeur typique	Impact
Mémoire RAM	2 Ko - 512 Ko	Protocoles légers requis
Mémoire Flash	32 Ko - 4 Mo	Code optimisé
Fréquence CPU	8 MHz - 240 MHz	Traitement limité
Consommation	10 μA - 500 mA	Gestion énergie critique
Coût unitaire	1$ - 50$	Production de masse

1.3 Les Microcontrôleurs pour l'IoT

1.3.1 Définition et Rôle

Un **microcontrôleur** (MCU - *Microcontroller Unit*) est un circuit intégré qui regroupe sur une même puce :

— Un processeur (CPU) pour l'exécution des instructions

— De la mémoire programme (Flash) et données (RAM/EEPROM)

— Des périphériques d'entrée/sortie (GPIO, ADC, DAC, PWM)

— Des interfaces de communication (UART, SPI, I2C)

— Des timers et contrôleurs d'interruption

1.3.2 Architecture Interne d'un Microcontrôleur

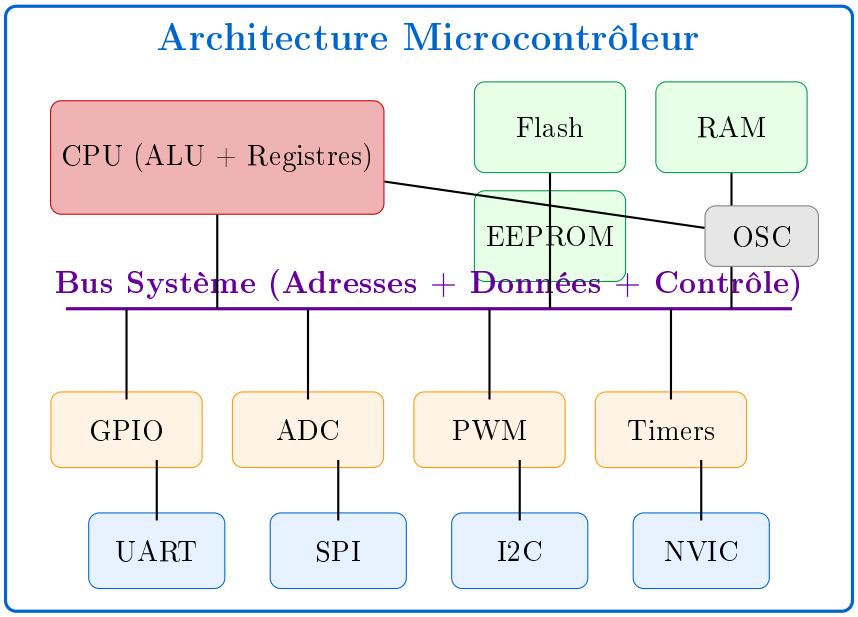


FIGURE 1.3 – Architecture interne d'un microcontrôleur générique

1.3.3 Plateformes de Développement IoT

Arduino

Arduino est une plateforme open-source basée sur des microcontrôleurs ATmega (AVR) ou ARM, très populaire pour le prototypage IoT.

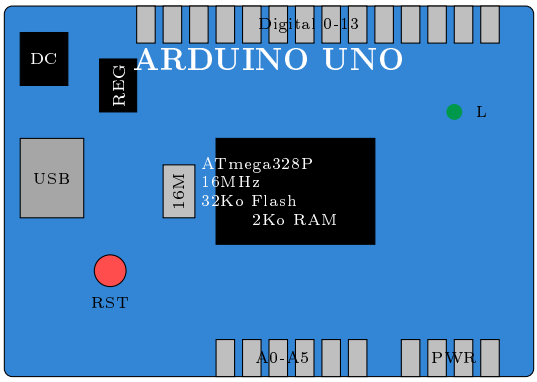


FIGURE 1.4 – Schéma de la carte Arduino Uno

Spécifications Arduino Uno :

— Microcontrôleur : ATmega328P (architecture AVR 8-bit)
— Fréquence : 16 MHz
— Mémoire Flash : 32 Ko (0,5 Ko bootloader)
— SRAM : 2 Ko
— EEPROM : 1 Ko
— GPIO : 14 digitaux (6 PWM), 6 analogiques
— Tension : 5V (entrée 7-12V)
— Courant par I/O : 20 mA (max 40 mA)

ESP32

L'ESP32 est un SoC (System on Chip) développé par Espressif, intégrant WiFi et Bluetooth, idéal pour l'IoT.



FIGURE 1.5 – Schéma de la carte ESP32 DevKit

Spécifications ESP32 :

— Processeur : Xtensa dual-core LX6 @ 240 MHz

— Mémoire : 520 Ko SRAM, 4 Mo Flash externe

— WiFi : 802.11 b/g/n (2.4 GHz)

— Bluetooth : Classic + BLE 4.2

— GPIO : 34 (dont 18 ADC, 2 DAC, 16 PWM)

— Interfaces : SPI, I2C, I2S, UART, CAN, Ethernet MAC

— Consommation : 5 μA (deep sleep) à 240 mA (TX WiFi)

— Sécurité : Secure Boot, Flash Encryption, Crypto accelerators

Raspberry Pi

Le Raspberry Pi est un micro-ordinateur monocarte sous Linux, offrant une puissance de calcul supérieure pour les applications IoT complexes.

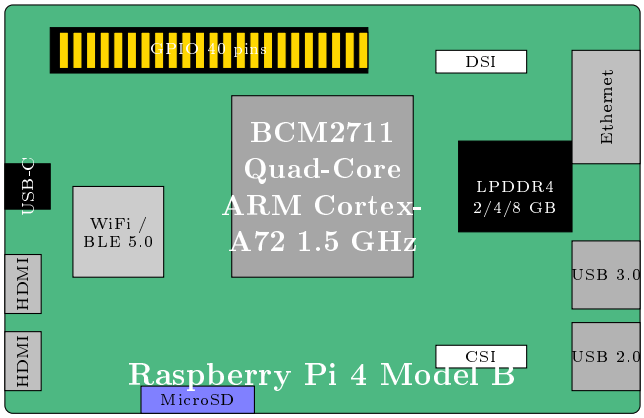


FIGURE 1.6 – Schéma de la carte Raspberry Pi 4

1.3.4 Tableau Comparatif des Plateformes

TABLE 1.3 – Comparaison des principales plateformes IoT

Caractéristique	Arduino Uno	ESP32	Raspberry Pi 4	STM32F4
Architecture	AVR 8-bit	Xtensa 32-bit	ARM Cortex-A72	ARM Cortex-M4
Fréquence	16 MHz	240 MHz	1.5 GHz	168 MHz
RAM	2 Ko	520 Ko	2-8 Go	192 Ko
Flash	32 Ko	4 Mo	MicroSD	1 Mo
WiFi intégré	Non	Oui	Oui	Non
Bluetooth	Non	Oui (BLE 4.2)	Oui (BLE 5.0)	Non
GPIO	20	34	40	82
ADC	6 (10-bit)	18 (12-bit)	Non natif	16 (12-bit)
OS	Bare-metal	FreeRTOS	Linux	RTOS/Bare-metal
Consommation	50 mA	80-240 mA	600-1200 mA	100-150 mA
Prix (approx.)	25€	10€	45-90€	15€
Cas d'usage	Prototypage	IoT connecté	Edge computing	Industriel

1.4 Les Capteurs

1.4.1 Principe de Fonctionnement

Un **capteur** est un transducteur qui convertit une grandeur physique mesurable (mesurande) en un signal électrique exploitable. La relation entre l'entrée et la sortie est caractérisée par la **fonction de transfert** :

$$V_{out} = f(M) = S \cdot M + V_0 \tag{1.2}$$

où :

— V_{out} : tension de sortie (V)

— M : mesurande (grandeur physique)

— S : sensibilité du capteur (V/unité)

— V_0 : offset (tension à mesurande nulle)

1.4.2 Chaîne d'Acquisition

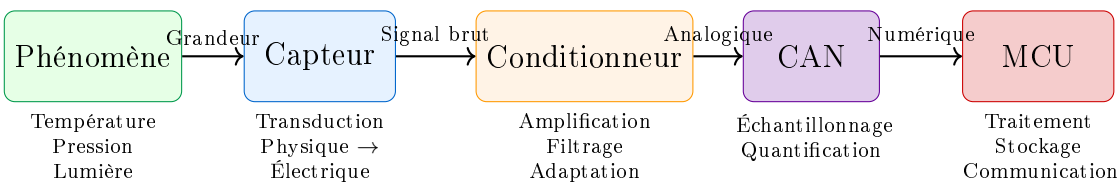


FIGURE 1.7 – Chaîne d'acquisition de données

1.4.3 Classification des Capteurs IoT

TABLE 1.4 – Classification détaillée des capteurs IoT

Type	Grandeur	Modèles	Plage	Précision	Interface
Température	°C	DS18B20	-55 à +125°C	±0.5°C	1-Wire
		DHT22	-40 à +80°C	±0.5°C	Digital
		BME280	-40 à +85°C	±1°C	I2C/SPI
		LM35	-55 à +150°C	±0.5°C	Analogique
Humidité	%RH	DHT22	0-100%	±2%	Digital
		BME280	0-100%	±3%	I2C/SPI
		HIH-4030	0-100%	±3.5%	Analogique
Pression	hPa	BMP280	300-1100 hPa	±1 hPa	I2C/SPI
		MPX5700	15-700 kPa	±2.5%	Analogique
Luminosité	Lux	BH1750	1-65535 lux	±20%	I2C
		TSL2561	0.1-40000 lux	Variable	I2C
		LDR	Relatif	–	Analogique
Distance	cm/m	HC-SR04	2-400 cm	±3 mm	Digital
		VL53L0X	0-200 cm	±3%	I2C
		LIDAR	0-40 m	±2 cm	UART/I2C
Mouvement	Présence	PIR HC-SR501	3-7 m	–	Digital
		RCWL-0516	5-9 m	–	Digital
Accélération	g	MPU6050	±2/4/8/16 g	0.06 mg	I2C
		ADXL345	±2/4/8/16 g	3.9 mg	I2C/SPI
Gaz	ppm	MQ-2	300-10000	±5%	Analogique
		CCS811	400-8192	±15%	I2C

1.4.4 Exemple : Capteur de Température DS18B20

Le DS18B20 est un capteur de température numérique utilisant le protocole 1-Wire, très répandu en IoT.

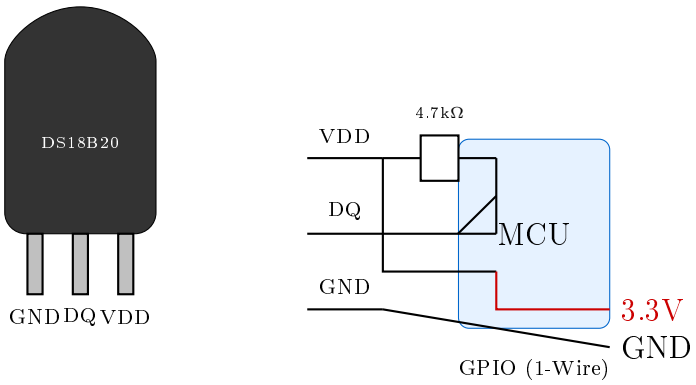


FIGURE 1.8 – Schéma de connexion du capteur DS18B20

```arduino
#include <OneWire.h>
#include <DallasTemperature.h>

#define ONE_WIRE_BUS 2  // Pin de donnees

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);
```

```
8
9  void setup() {
10     Serial.begin(9600);
11     sensors.begin();
12 }
13
14 void loop() {
15     sensors.requestTemperatures();
16     float tempC = sensors.getTempCByIndex(0);
17
18     if (tempC != DEVICE_DISCONNECTED_C) {
19         Serial.print("Temperature: ");
20         Serial.print(tempC);
21         Serial.println(" C");
22     }
23
24     delay(1000);
25 }
```

Listing 1.1 – Lecture du capteur DS18B20 sur Arduino

1.5 Les Actionneurs

1.5.1 Définition et Classification

Un **actionneur** est un dispositif qui convertit un signal électrique de commande en une action physique sur l'environnement. Il constitue l'élément effecteur du système IoT, permettant d'agir sur le monde réel.

TABLE 1.5 – Classification des actionneurs IoT

Type	Exemples	Commande	Application
Électromagnétique	Relais, Solénoïdes	Digital (ON/OFF)	Commutation puissance
Moteurs DC	Moteurs à courant continu	PWM (vitesse)	Ventilation, mobilité
Servomoteurs	SG90, MG996R	PWM (position)	Robotique, orientation
Moteurs pas à pas	NEMA 17, 28BYJ-48	Séquence impulsions	Positionnement précis
Optique	LED, Afficheurs	Digital/PWM	Signalisation, IHM
Acoustique	Buzzers, Haut-parleurs	Digital/PWM	Alertes sonores
Thermique	Résistances chauffantes	PWM/Relais	Régulation température
Hydraulique	Électrovannes	Digital	Irrigation, fluides

1.5.2 Commande PWM

La **Modulation de Largeur d'Impulsion** (PWM - *Pulse Width Modulation*) permet de contrôler la puissance moyenne fournie à un actionneur en faisant varier le rapport cyclique d'un signal carré.

$$P_{moyenne} = P_{max} \times \frac{t_{ON}}{T} = P_{max} \times D \tag{1.3}$$

où $D = \frac{t_{ON}}{T}$ est le rapport cyclique (duty cycle) compris entre 0 et 1.

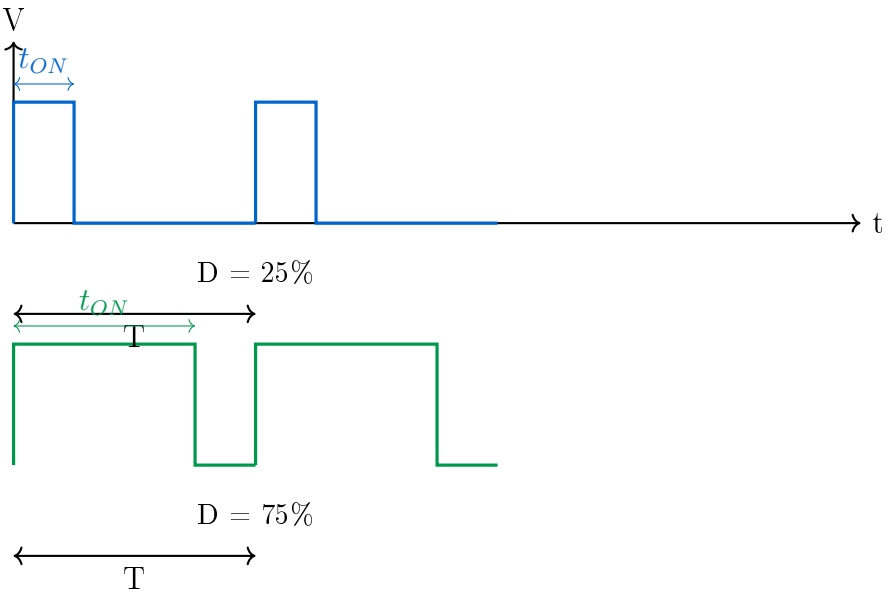


FIGURE 1.9 – Signaux PWM avec différents rapports cycliques

1.6 Domaines d'Application de l'IoT

L'IoT s'étend à de nombreux secteurs d'activité, chacun avec ses spécificités et ses cas d'usage particuliers.

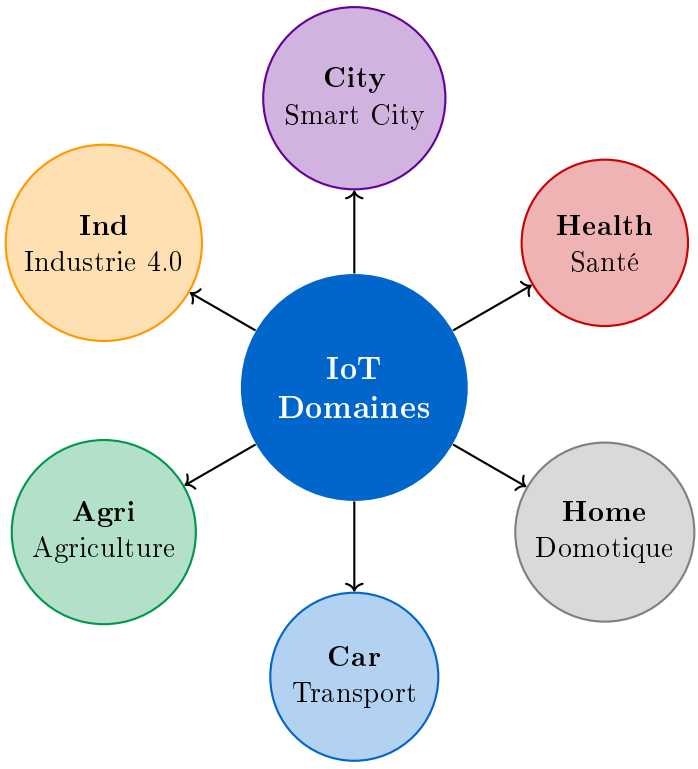


FIGURE 1.10 – Principaux domaines d'application de l'IoT

1.6.1 Smart City (Ville Intelligente)

La ville intelligente intègre les technologies IoT pour optimiser la gestion urbaine et améliorer la qualité de vie des citoyens.

Applications principales :

— **Éclairage intelligent** : adaptation automatique à la luminosité ambiante et à la présence de piétons, réduction de 30-50% de la consommation énergétique

— **Gestion du trafic** : feux adaptatifs basés sur le flux réel, détection de congestion, optimisation des itinéraires

— **Stationnement intelligent** : capteurs de places disponibles, guidage des automobilistes, paiement dématérialisé

— **Gestion des déchets** : poubelles connectées mesurant le taux de remplissage, optimisation des tournées de collecte

— **Qualité environnementale** : réseau de capteurs pour la qualité de l'air (PM2.5, CO2, NOx), bruit, température

1.6.2 Industrie 4.0

L'Industrie 4.0 représente la quatrième révolution industrielle, caractérisée par l'intégration des technologies numériques (IoT, IA, Big Data) dans les processus de production.

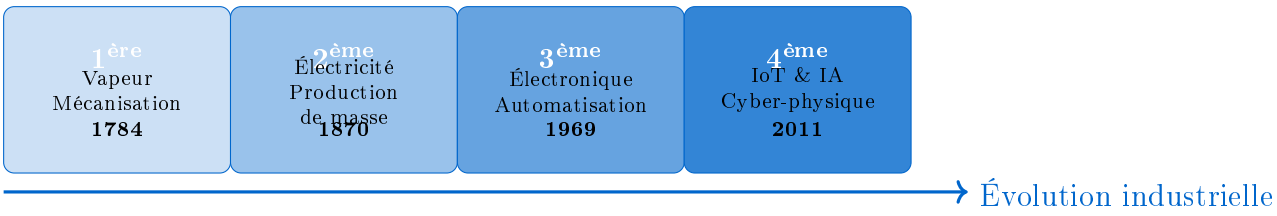


FIGURE 1.11 – Les quatre révolutions industrielles

1.6.3 Santé Connectée (e-Health)

L'IoT médical permet le suivi à distance des patients, la détection précoce des anomalies et l'amélioration de la qualité des soins.

Dispositifs et applications :

— **Wearables** : montres connectées (ECG, SpO2), bracelets d'activité

— **Dispositifs médicaux** : glucomètres connectés, tensiomètres, balances

— **Implants intelligents** : pacemakers connectés, pompes à insuline

— **Télémédecine** : consultations à distance, suivi post-opératoire

1.6.4 Agriculture de Précision

L'agriculture connectée utilise les capteurs et l'analyse de données pour optimiser les rendements tout en réduisant l'impact environnemental.

Technologies déployées :

— Capteurs sol : humidité, pH, nutriments (NPK)

— Stations météo connectées : température, pluviométrie, vent

— Imagerie par drone : NDVI (indice de végétation), détection maladies
— Irrigation automatisée : goutte-à-goutte piloté par capteurs
— Traçabilité : RFID pour le suivi du bétail et des récoltes

Chapitre 2

Pile de Protocoles de Communication IoT

2.1 Introduction

La communication dans l'IoT repose sur une pile protocolaire adaptée aux contraintes spécifiques des objets connectés : faible consommation énergétique, bande passante limitée, latence variable et hétérogénéité des dispositifs. Cette pile diffère significativement du modèle OSI traditionnel pour répondre aux exigences des réseaux de capteurs.

2.1.1 Modèle de Référence IoT

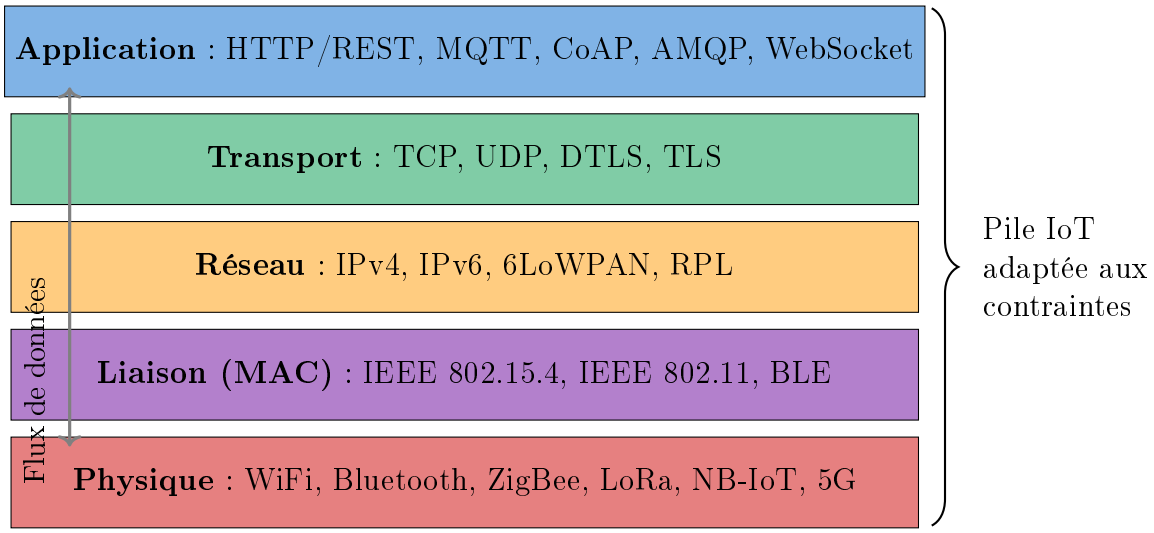


FIGURE 2.1 – Pile protocolaire IoT

2.2 Couche Physique

La couche physique définit les caractéristiques de transmission radio ou filaire des données. Dans l'IoT, plusieurs technologies coexistent selon les besoins en portée, débit et consommation.

2.2.1 Technologies Sans Fil à Courte Portée

WiFi (IEEE 802.11)

Le WiFi reste la technologie de référence pour les objets nécessitant un haut débit et une connexion Internet directe.

TABLE 2.1 – Caractéristiques des normes WiFi pour l'IoT

Norme	Fréquence	Débit max	Portée	Conso.	Usage IoT
802.11b	2.4 GHz	11 Mbps	35 m	Élevée	Legacy
802.11g	2.4 GHz	54 Mbps	38 m	Élevée	Courant
802.11n	2.4/5 GHz	600 Mbps	70 m	Moyenne	Recommandé
802.11ac	5 GHz	1.3 Gbps	35 m	Élevée	Streaming
802.11ah (HaLow)	**900 MHz**	**347 Mbps**	**1 km**	**Faible**	**IoT spécifique**
802.11ax (WiFi 6)	2.4/5/6 GHz	9.6 Gbps	35 m	Variable	Dense IoT

Bluetooth Low Energy (BLE)

Le BLE est optimisé pour l'IoT avec une consommation réduite, idéal pour les wearables et capteurs alimentés par batterie.

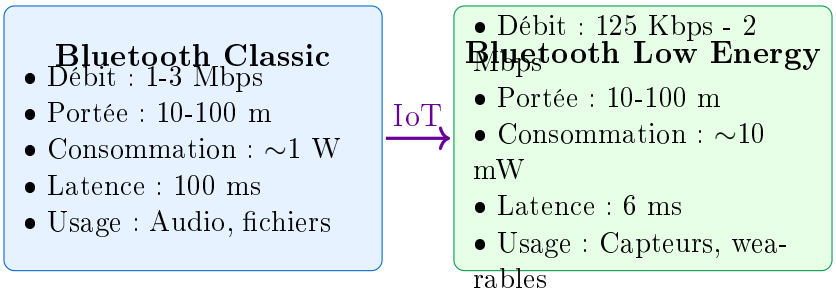


FIGURE 2.2 – Comparaison Bluetooth Classic vs BLE

Architecture BLE :

— **Advertising** : diffusion périodique pour la découverte

— **GATT** (Generic Attribute Profile) : échange de données structurées

— **Services et Caractéristiques** : modèle de données hiérarchique

— **Modes** : Peripheral (esclave), Central (maître), Broadcaster, Observer

ZigBee (IEEE 802.15.4)

ZigBee est un protocole de communication basse consommation conçu pour les réseaux de capteurs maillés.

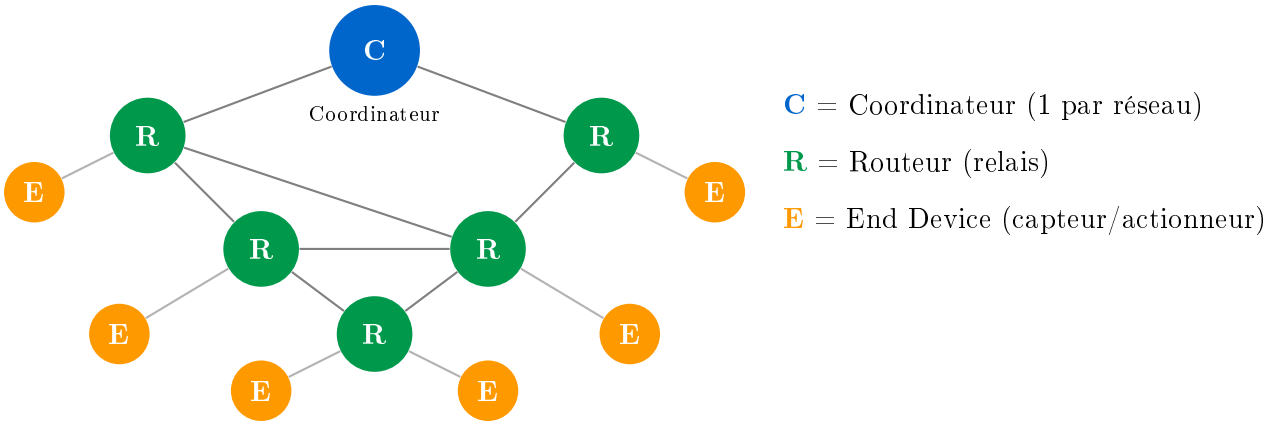


FIGURE 2.3 – Topologie maillée ZigBee

Caractéristiques ZigBee :

— Fréquence : 2.4 GHz (mondial), 868 MHz (Europe), 915 MHz (USA)

— Débit : 250 kbps (2.4 GHz), 20-40 kbps (sub-GHz)

— Portée : 10-100 m (extensible par maillage)

— Jusqu'à 65 536 nœuds par réseau

— Consommation en veille : $< 1\ \mu$A

2.2.2 Technologies LPWAN (Low Power Wide Area Network)

Les technologies LPWAN sont conçues pour les communications longue portée avec une très faible consommation.

LoRa et LoRaWAN

LoRa (Long Range) est une technologie de modulation radio propriétaire (Semtech) permettant des communications jusqu'à 15 km en zone rurale.

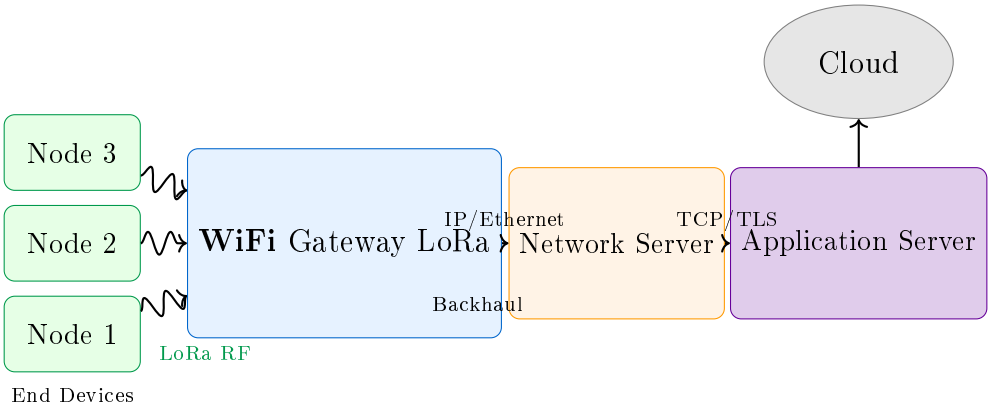


FIGURE 2.4 – Architecture LoRaWAN

Classes de dispositifs LoRaWAN :

TABLE 2.2 – Classes de dispositifs LoRaWAN

Classe	Réception	Latence DL	Consommation	Usage
Classe A	Après émission uniquement	Haute (secondes)	Très faible	Capteurs batterie
Classe B	Fenêtres planifiées (beacon)	Moyenne	Faible	Actionneurs
Classe C	Réception continue	Très faible (ms)	Élevée	Alimentation secteur

Paramètres de modulation LoRa :

Le débit et la portée dépendent du *Spreading Factor* (SF) :

$$R_b = SF \times \frac{BW}{2^{SF}} \times CR \tag{2.1}$$

où BW est la bande passante (125/250/500 kHz) et CR le taux de codage (4/5 à 4/8).

TABLE 2.3 – Impact du Spreading Factor sur les performances LoRa

SF	Débit (bps)	Sensibilité (dBm)	Portée approx.	Temps sur l'air
SF7	5 470	-123	2 km	Court
SF8	3 125	-126	4 km	× 2
SF9	1 760	-129	6 km	× 4
SF10	980	-132	8 km	× 8
SF11	440	-134.5	11 km	× 16
SF12	250	-137	15 km	× 32

Technologies Cellulaires IoT

TABLE 2.4 – Comparaison des technologies cellulaires IoT

Technologie	Débit DL	Débit UL	Latence	Portée	Mobilité
NB-IoT	250 kbps	250 kbps	1.6-10 s	10 km	Limitée
LTE-M (Cat-M1)	1 Mbps	1 Mbps	10-15 ms	11 km	Oui
4G LTE	150 Mbps	50 Mbps	10-50 ms	5 km	Oui
5G mMTC	10 Mbps	10 Mbps	1-10 ms	1-10 km	Oui
5G URLLC	1 Gbps	500 Mbps	$<$1 ms	1 km	Oui

2.2.3 Comparaison Globale des Technologies

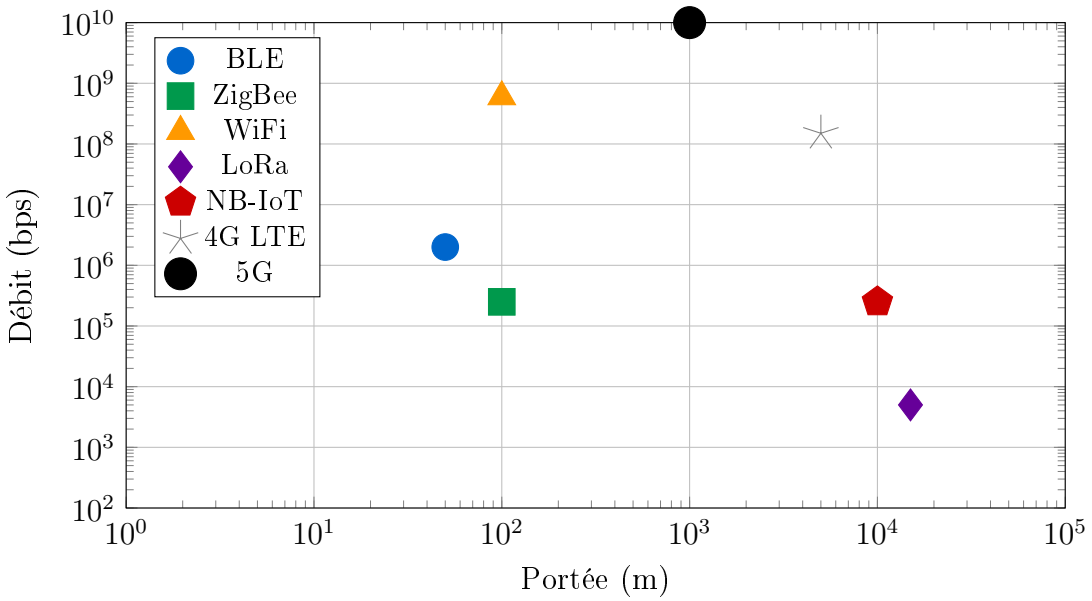


FIGURE 2.5 – Positionnement des technologies sans fil IoT (portée vs débit)

2.3 Couche Réseau

2.3.1 IPv6 et 6LoWPAN

6LoWPAN (IPv6 over Low-Power Wireless Personal Area Networks) est une couche d'adaptation permettant de transporter IPv6 sur des réseaux contraints IEEE 802.15.4.

Mécanismes principaux :

— **Compression d'en-tête** : réduction de 40 octets (IPv6) à 2-7 octets

— **Fragmentation** : adaptation au MTU de 127 octets (802.15.4)

— **Mesh addressing** : routage au niveau liaison

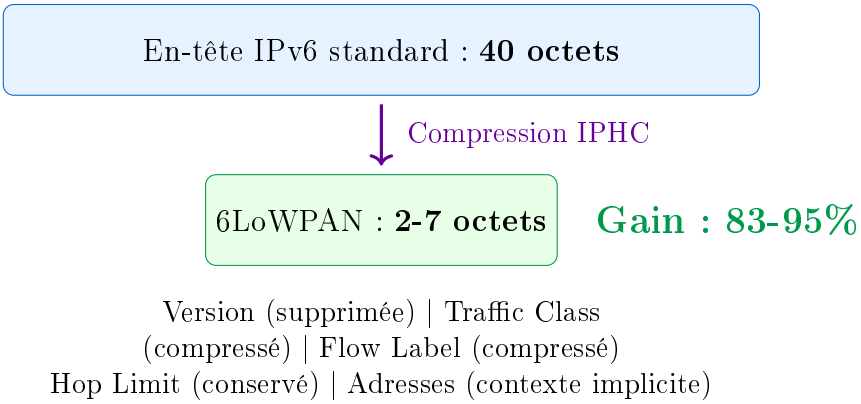


FIGURE 2.6 – Compression d'en-tête 6LoWPAN

2.3.2 Protocole RPL

RPL (Routing Protocol for Low-Power and Lossy Networks) est le protocole de routage standard pour les réseaux IoT (RFC 6550).

Caractéristiques :

— Topologie : DODAG (Destination-Oriented Directed Acyclic Graph)

— Métrique : ETX (Expected Transmission Count), énergie, latence

— Messages : DIO (Information), DIS (Solicitation), DAO (Advertisement)

— Support multipoint-to-point, point-to-multipoint, point-to-point

2.4 Couche Transport

2.4.1 TCP vs UDP pour l'IoT

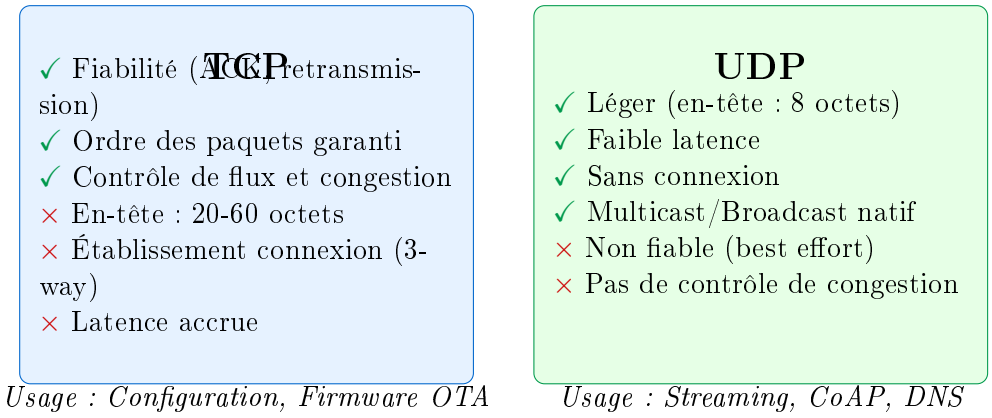


FIGURE 2.7 – Comparaison TCP vs UDP pour l'IoT

2.5 Couche Application

2.5.1 MQTT (Message Queuing Telemetry Transport)

MQTT est le protocole de messagerie de référence pour l'IoT, basé sur le modèle *publish/subscribe*.

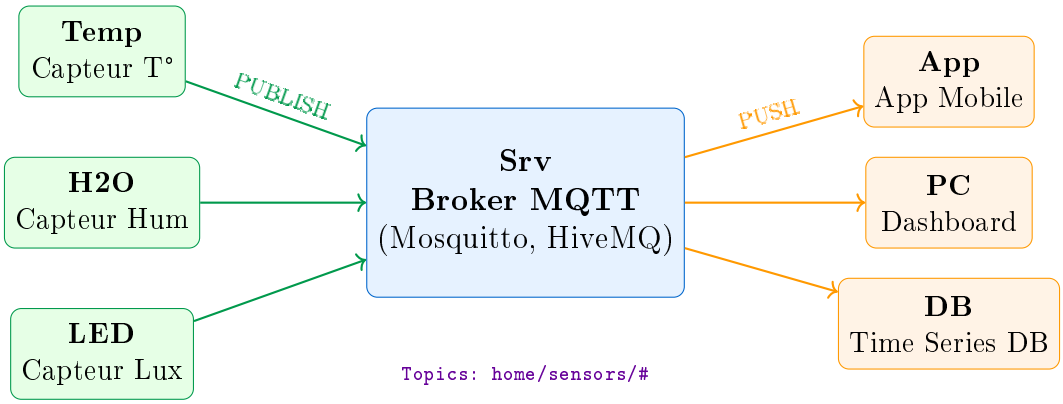


FIGURE 2.8 – Architecture Publish/Subscribe MQTT

Structure d'un topic MQTT :

```
building/floor1/room101/temperature
home/+/humidity          (+ = wildcard simple niveau)
factory/machine/#        (# = wildcard multi-niveaux)
```

Niveaux de QoS (Quality of Service) :

TABLE 2.5 – Niveaux de QoS MQTT

QoS	Nom	Garantie	Messages	Usage
0	At most once	Aucune (fire & forget)	PUBLISH	Télémétrie fréquente
1	At least once	Au moins 1 réception	PUBLISH + PUBACK	Alertes
2	Exactly once	Exactement 1 réception	4 messages	Transactions critiques

```python
import paho.mqtt.client as mqtt
import json
import time

# Callbacks
def on_connect(client, userdata, flags, rc):
    print(f"Connected with result code {rc}")
    client.subscribe("sensors/#")

def on_message(client, userdata, msg):
    data = json.loads(msg.payload.decode())
    print(f"Topic: {msg.topic}")
    print(f"Data: {data}")

# Configuration client
client = mqtt.Client(client_id="iot_subscriber")
client.on_connect = on_connect
client.on_message = on_message

# Connexion avec authentification
client.username_pw_set("user", "password")
client.tls_set()  # Activer TLS
client.connect("broker.example.com", 8883, 60)

# Publication
def publish_sensor_data(temperature, humidity):
    payload = json.dumps({
        "temperature": temperature,
        "humidity": humidity,
        "timestamp": time.time()
    })
    client.publish("sensors/room1", payload, qos=1)

client.loop_forever()
```

Listing 2.1 – Exemple client MQTT en Python (paho-mqtt)

2.5.2 CoAP (Constrained Application Protocol)

CoAP est un protocole RESTful conçu pour les dispositifs très contraints, utilisant UDP.

Caractéristiques principales :

— Modèle REST : GET, POST, PUT, DELETE

— En-tête minimal : 4 octets fixes

— Transport : UDP (avec DTLS pour la sécurité)

— Observation : notification push des ressources

— Découverte : /.well-known/core

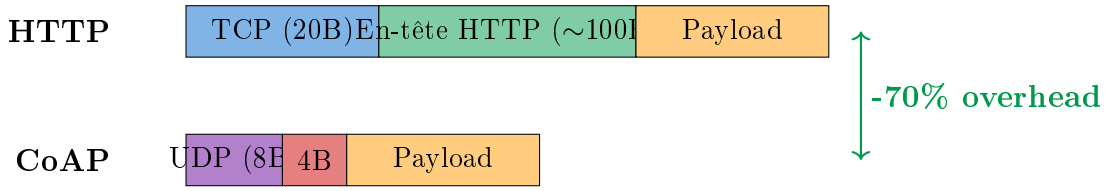


FIGURE 2.9 – Comparaison overhead HTTP vs CoAP

2.5.3 Tableau Comparatif des Protocoles Applicatifs

TABLE 2.6 – Comparaison des protocoles applicatifs IoT

Critère	HTTP	MQTT	CoAP	AMQP	WebSocket
Transport	TCP	TCP	UDP	TCP	TCP
Modèle	Request/Response	Pub/Sub	Request/Response	Pub/Sub + Queue	Bidirectionnel
En-tête min	~100 B	2 B	4 B	8 B	2 B
QoS	Non	0, 1, 2	Con/Non-Con	Oui	Non
Sécurité	TLS	TLS	DTLS	TLS/SASL	TLS
Broker requis	Non	Oui	Non	Oui	Non
Push natif	Non	Oui	Observe	Oui	Oui
Cas d'usage	Web, API	Télémétrie	Contraint	Enterprise	Temps réel

Chapitre 3

Niveaux de Déploiement : Edge, Fog et Cloud

3.1 Introduction

L'architecture de déploiement IoT moderne repose sur trois niveaux complémentaires qui permettent d'optimiser le traitement des données en fonction des contraintes de latence, bande passante et capacité de calcul.

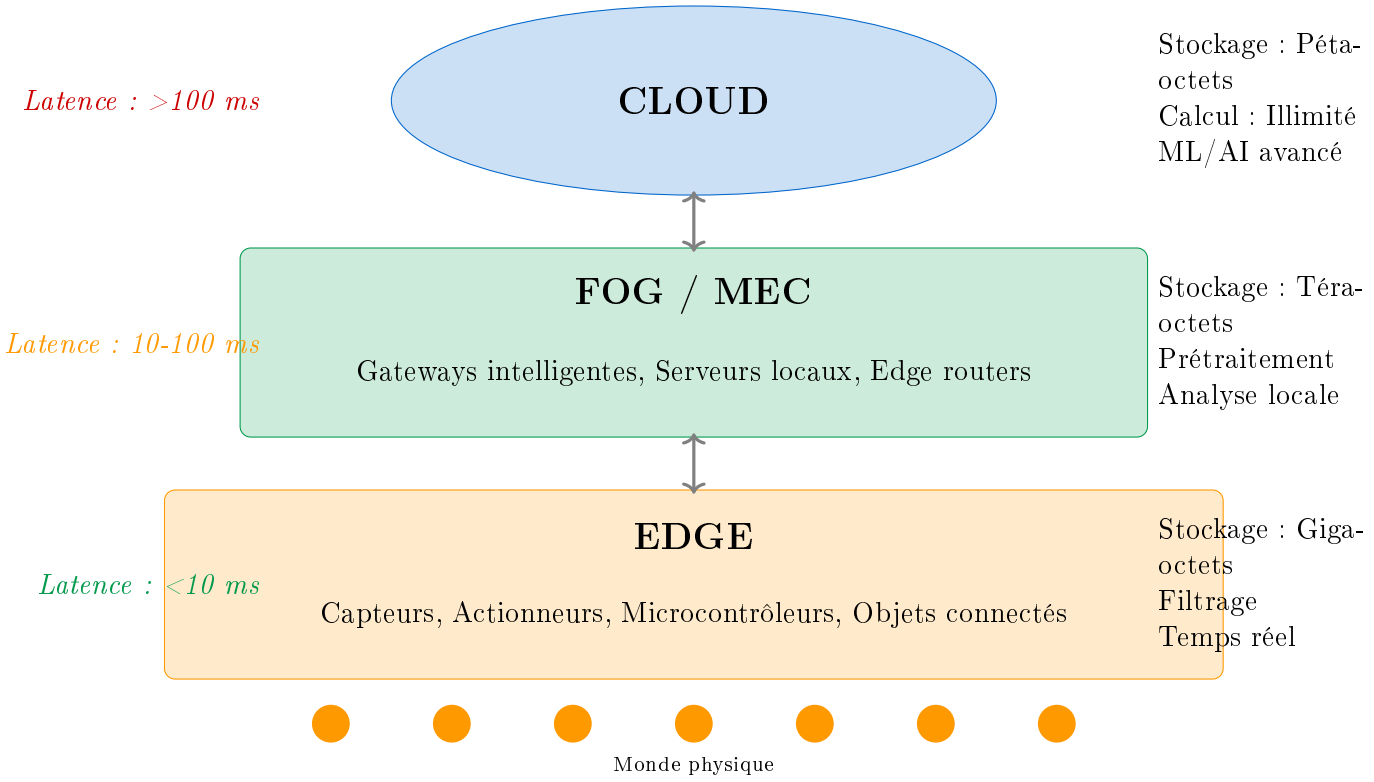


FIGURE 3.1 – Architecture Edge-Fog-Cloud

3.2 Edge Computing

3.2.1 Définition

L'**Edge Computing** (ou informatique en périphérie) désigne le traitement des données au plus près de leur source de génération, directement sur les dispositifs IoT ou sur des équipements à proximité immédiate.

3.2.2 Avantages

1. **Latence ultra-faible** : traitement en millisecondes pour les applications temps réel
2. **Réduction de la bande passante** : seules les données pertinentes sont transmises
3. **Autonomie** : fonctionnement même en cas de perte de connectivité
4. **Confidentialité** : données sensibles traitées localement
5. **Scalabilité** : distribution de la charge de calcul

3.2.3 Technologies Edge

TABLE 3.1 – Plateformes Edge Computing

Plateforme	Fournisseur	Caractéristiques	Cas d'usage
AWS Greengrass	Amazon	Lambda local, ML inference	Industrie, véhicules
Azure IoT Edge	Microsoft	Conteneurs, AI	Smart building
EdgeX Foundry	Linux Foundation	Open source, microservices	Interopérabilité
KubeEdge	CNCF	Kubernetes à l'edge	Cloud-native IoT

3.3 Fog Computing

3.3.1 Définition

Le **Fog Computing** (ou informatique en brouillard) constitue une couche intermédiaire entre l'Edge et le Cloud, offrant des capacités de calcul, stockage et réseau distribuées géographiquement.

3.3.2 Fonctions du Fog

— **Agrégation** : consolidation des données de multiples capteurs

— **Filtrage** : élimination des données redondantes ou erronées

— **Compression** : réduction du volume avant envoi vers le Cloud

— **Cache** : stockage temporaire pour accès rapide

— **Analyse locale** : détection de patterns, alertes immédiates

— **Orchestration** : coordination des dispositifs Edge

3.4 Cloud Computing pour l'IoT

3.4.1 Services Cloud

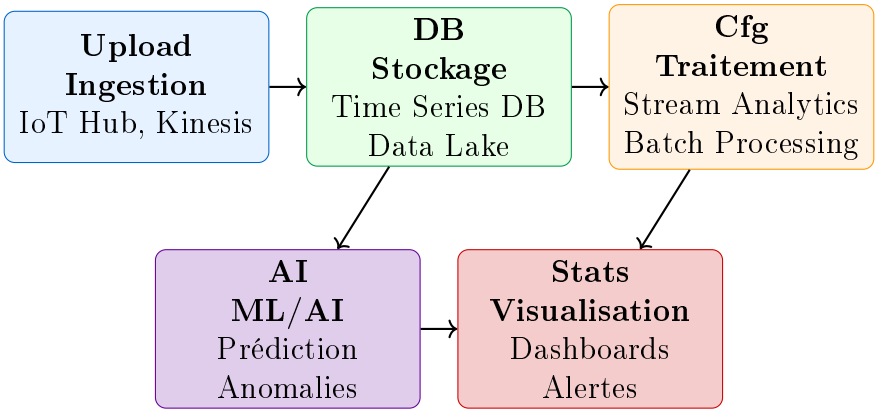


FIGURE 3.2 – Pipeline de données IoT dans le Cloud

3.4.2 Comparaison des Trois Niveaux

TABLE 3.2 – Comparaison Edge vs Fog vs Cloud

Critère	Edge	Fog	Cloud
Latence	<10 ms	10-100 ms	>100 ms
Capacité calcul	Limitée (MCU)	Modérée (Gateway)	Illimitée
Stockage	Ko-Go	Go-To	Po+
Disponibilité	Variable	Haute	Très haute (99.99%)
Sécurité physique	Faible	Moyenne	Élevée
Coût unitaire	Faible	Moyen	Élevé (usage)
Mise à l'échelle	Difficile	Régionale	Globale
Connectivité requise	Non	Intermittente	Continue

Chapitre 4

Sécurité de l'Internet des Objets

4.1 Introduction et Enjeux

La sécurité constitue un défi majeur de l'IoT en raison des contraintes des dispositifs (ressources limitées, déploiement massif) et de la surface d'attaque étendue. Les incidents de sécurité IoT peuvent avoir des conséquences graves : atteinte à la vie privée, sabotage industriel, voire mise en danger de vies humaines.

4.1.1 Menaces Principales

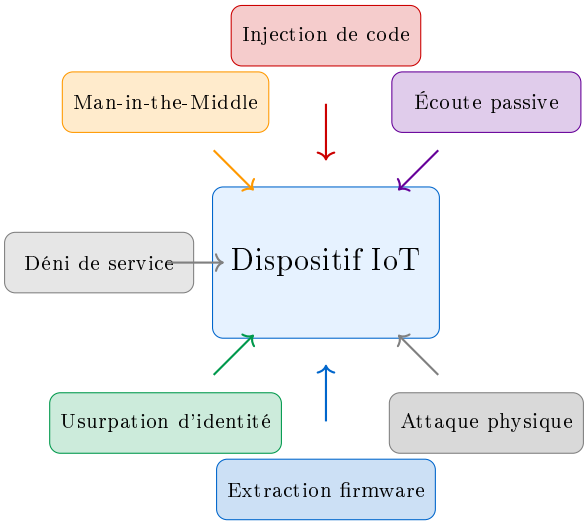


FIGURE 4.1 – Surface d'attaque d'un dispositif IoT

4.2 Sécurité par Couche

4.2.1 Couche Physique

Menaces :

— Accès physique non autorisé
— Extraction de clés depuis la mémoire
— Attaques par canaux auxiliaires (side-channel)

— Clonage de dispositifs

Contre-mesures :

— Boîtiers inviolables (tamper-evident/tamper-resistant)
— Secure Element / TPM (Trusted Platform Module)
— PUF (Physically Unclonable Functions)
— Effacement sécurisé des clés en cas d'intrusion

4.2.2 Couche Communication

Protocoles de sécurité :

TABLE 4.1 – Protocoles de sécurité pour l'IoT

Couche	Protocole	Fonction	Overhead
Transport	TLS 1.3	Chiffrement TCP	Modéré
Transport	DTLS 1.2/1.3	Chiffrement UDP	Modéré
Réseau	IPsec	VPN, tunnel sécurisé	Élevé
Application	MQTT + TLS	Messagerie sécurisée	Faible
Application	CoAP + DTLS	REST sécurisé	Faible
Liaison	IEEE 802.15.4 (AES-CCM)	Chiffrement ZigBee	Très faible

4.2.3 Authentification et Autorisation

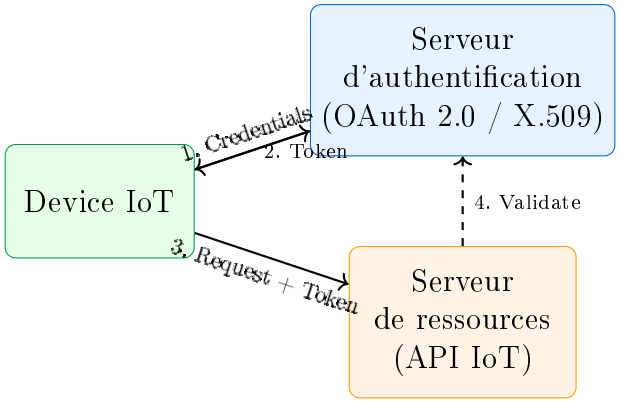


FIGURE 4.2 – Flux d'authentification OAuth 2.0 pour IoT

4.3 Bonnes Pratiques de Sécurité IoT

1. **Sécurité by Design** : intégrer la sécurité dès la conception
2. **Principe du moindre privilège** : limiter les accès au strict nécessaire
3. **Mise à jour sécurisée** : OTA (Over-The-Air) avec signature cryptographique
4. **Chiffrement des données** : au repos et en transit
5. **Gestion des clés** : rotation régulière, stockage sécurisé
6. **Monitoring** : détection d'anomalies et intrusions
7. **Segmentation réseau** : isolation des dispositifs IoT

Chapitre 5

Projets Pratiques IoT

5.1 Projet 1 : Station Météo Connectée

5.1.1 Objectifs Pédagogiques

— Interfacer des capteurs environnementaux
— Implémenter la communication MQTT
— Visualiser les données sur un dashboard

5.1.2 Matériel Requis

— ESP32 DevKit
— Capteur BME280 (température, humidité, pression)
— Capteur de luminosité BH1750
— Alimentation USB ou batterie LiPo

5.1.3 Schéma de Câblage

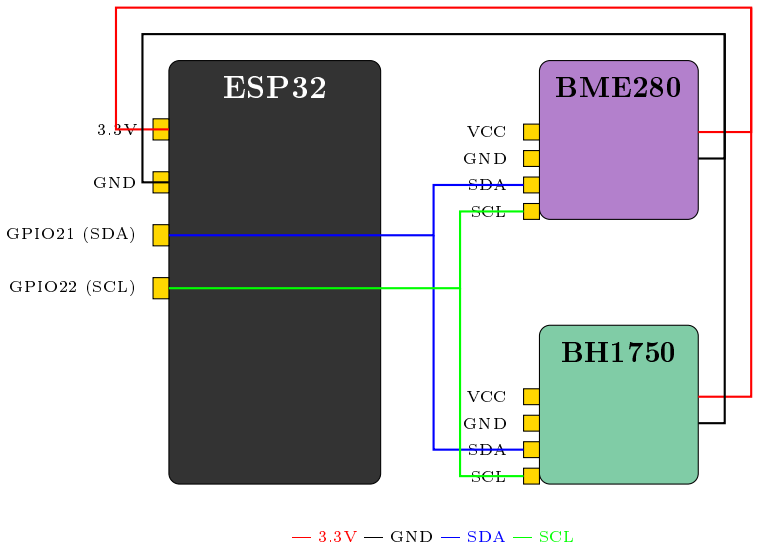


FIGURE 5.1 – Schéma de câblage de la station météo

5.1.4 Code Source

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <Wire.h>
#include <Adafruit_BME280.h>
#include <BH1750.h>
#include <ArduinoJson.h>

// Configuration WiFi et MQTT
const char* ssid = "VotreSSID";
const char* password = "VotreMotDePasse";
const char* mqtt_server = "broker.hivemq.com";
const int mqtt_port = 1883;
const char* mqtt_topic = "iot/station_meteo/data";

// Objets capteurs
Adafruit_BME280 bme;
BH1750 lightMeter;

WiFiClient espClient;
PubSubClient client(espClient);

void setup_wifi() {
    delay(10);
    Serial.println("Connexion␣WiFi...");
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\nWiFi␣connecte!");
    Serial.println(WiFi.localIP());
}

void reconnect() {
    while (!client.connected()) {
        Serial.print("Connexion␣MQTT...");
        String clientId = "ESP32_Meteo_" + String(random(0xffff),
           HEX);
        if (client.connect(clientId.c_str())) {
            Serial.println("connecte");
        } else {
            Serial.print("echec,␣rc=");
            Serial.print(client.state());
            delay(5000);
        }
    }
}

void setup() {
```

```
49      Serial.begin(115200);
50      Wire.begin(21, 22);  // SDA, SCL
51
52      // Initialisation BME280
53      if (!bme.begin(0x76)) {
54          Serial.println("BME280 non trouve!");
55          while (1);
56      }
57
58      // Initialisation BH1750
59      if (!lightMeter.begin()) {
60          Serial.println("BH1750 non trouve!");
61          while (1);
62      }
63
64      setup_wifi();
65      client.setServer(mqtt_server, mqtt_port);
66  }
67
68  void loop() {
69      if (!client.connected()) {
70          reconnect();
71      }
72      client.loop();
73
74      // Lecture des capteurs
75      float temperature = bme.readTemperature();
76      float humidity = bme.readHumidity();
77      float pressure = bme.readPressure() / 100.0F;
78      float lux = lightMeter.readLightLevel();
79
80      // Creation du JSON
81      StaticJsonDocument<256> doc;
82      doc["device_id"] = "station_01";
83      doc["temperature"] = temperature;
84      doc["humidity"] = humidity;
85      doc["pressure"] = pressure;
86      doc["luminosity"] = lux;
87      doc["timestamp"] = millis();
88
89      char buffer[256];
90      serializeJson(doc, buffer);
91
92      // Publication MQTT
93      client.publish(mqtt_topic, buffer);
94
95      Serial.printf("T: %.1fC, H: %.1f%%, P: %.1fhPa, L: %.1flux\n",
96                    temperature, humidity, pressure, lux);
97
98      delay(60000);  // Envoi toutes les minutes
99  }
```

Listing 5.1 – Station météo connectée (ESP32 + MQTT)

5.2 Projet 2 : Système de Contrôle d'Accès RFID

5.2.1 Objectifs

— Lecture de badges RFID/NFC
— Gestion d'une base d'utilisateurs autorisés
— Contrôle d'un actionneur (gâche électrique)
— Journalisation des accès

5.2.2 Matériel

— Arduino Uno ou ESP32
— Module RFID RC522 (MFRC522)
— Relais 5V pour gâche électrique
— LED RGB pour indication d'état
— Buzzer pour feedback sonore

5.3 Projet 3 : Système d'Irrigation Intelligent

5.3.1 Architecture du Système

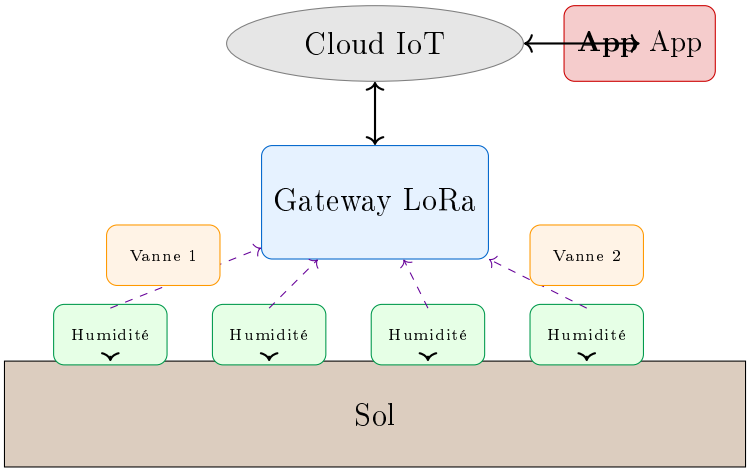


FIGURE 5.2 – Architecture du système d'irrigation intelligent

Chapitre 6

Intelligence Artificielle et Machine Learning pour l'IoT

6.1 Introduction à l'IA pour l'IoT

L'intégration de l'Intelligence Artificielle (IA) et du Machine Learning (ML) dans les systèmes IoT représente une évolution majeure qui transforme les données brutes collectées par les capteurs en informations actionnables et en décisions automatisées.

6.1.1 Convergence IoT et IA

La convergence de l'IoT et de l'IA, parfois appelée **AIoT** (Artificial Intelligence of Things), permet de :

— **Analyser** des volumes massifs de données en temps réel

— **Prédire** des événements futurs (maintenance prédictive, prévisions météo)

— **Détecter** des anomalies et des comportements suspects

— **Optimiser** les processus et la consommation énergétique

— **Personnaliser** les services en fonction du contexte utilisateur

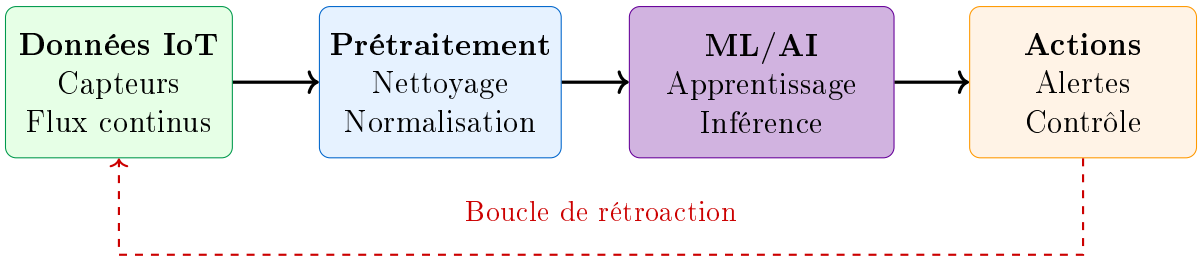


FIGURE 6.1 – Pipeline de traitement IA pour l'IoT

6.1.2 Types d'Apprentissage Automatique

TABLE 6.1 – Types d'apprentissage automatique appliqués à l'IoT

Type	Principe	Algorithmes	Applications IoT
Supervisé	Apprentissage avec labels	Régression, SVM, Random Forest, CNN	Classification de défauts, prédiction
Non supervisé	Sans labels	K-means, DBSCAN, Autoencoders	Détection d'anomalies, clustering de capteurs
Par renforcement	Récompenses/punitions	Q-Learning, DQN, PPO	Contrôle robotique, optimisation énergie
Semi-supervisé	Labels partiels	Self-training, Co-training	Labellisation auto, expansion de datasets

6.2 TinyML : Machine Learning Embarqué

6.2.1 Définition et Enjeux

Le **TinyML** (Tiny Machine Learning) désigne l'exécution d'algorithmes de machine learning directement sur des microcontrôleurs à ressources très limitées (quelques Ko de RAM, quelques MHz de fréquence).

$$\text{Contraintes TinyML} : \begin{cases} \text{RAM} < 256 \text{ Ko} \\ \text{Flash} < 1 \text{ Mo} \\ \text{Puissance} < 1 \text{ mW} \\ \text{Latence} < 100 \text{ ms} \end{cases} \tag{6.1}$$

6.2.2 Frameworks TinyML

TABLE 6.2 – Principaux frameworks TinyML

Framework	Développeur	Cibles	Caractéristiques
TensorFlow Lite Micro	Google	ARM Cortex-M, ESP32	Large écosystème
Edge Impulse	Edge Impulse Inc.	Multi-plateforme	Studio visuel, AutoML
STM32Cube.AI	STMicroelectronics	STM32	Intégration native
CMSIS-NN	ARM	Cortex-M	Optimisé ARM
microTVM	Apache TVM	Multi-cible	Compilation optimisée

6.2.3 Pipeline de Développement TinyML

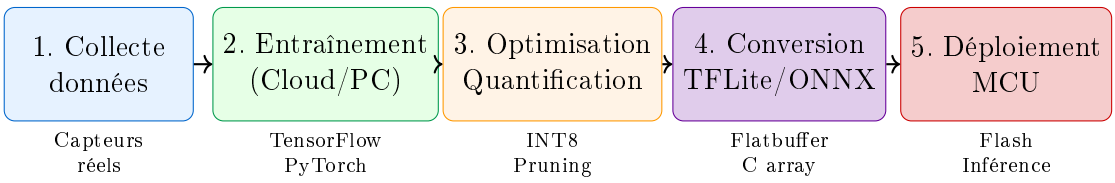


FIGURE 6.2 – Pipeline de développement TinyML

6.2.4 Techniques d'Optimisation des Modèles

Pour faire tenir des modèles de deep learning sur des microcontrôleurs, plusieurs techniques d'optimisation sont employées :

Quantification

La **quantification** réduit la précision des poids et activations du réseau de neurones :

TABLE 6.3 – Impact de la quantification

Type	Bits	Taille	Vitesse	Précision
Float32 (original)	32	100%	1x	100%
Float16	16	50%	1.5x	99.9%
INT8	8	25%	3-4x	98-99%
INT4	4	12.5%	5-8x	95-98%
Binaire	1	3%	10-50x	85-95%

Pruning (Élagage)

Le **pruning** supprime les connexions (poids) les moins importantes du réseau :

$$\text{Poids élagués : } w_i = 0 \text{ si } |w_i| < \theta \tag{6.2}$$

où θ est un seuil déterminé par la sensibilité du modèle.

Knowledge Distillation

La **distillation de connaissances** transfère le savoir d'un grand modèle (teacher) vers un modèle plus petit (student) :

$$\mathcal{L}_{distill} = \alpha \cdot \mathcal{L}_{CE}(y, \hat{y}_{student}) + (1 - \alpha) \cdot \mathcal{L}_{KL}(\hat{y}_{teacher}, \hat{y}_{student}) \tag{6.3}$$

6.2.5 Exemple : Classification Audio sur Arduino

```
#include <TensorFlowLite.h>
#include <tensorflow/lite/micro/all_ops_resolver.h>
#include <tensorflow/lite/micro/micro_interpreter.h>

// Mod le quantifi  (g  n  r  par xxd)
#include "model_data.h"

// Configuration
constexpr int kTensorArenaSize = 10 * 1024;  // 10 Ko
uint8_t tensor_arena[kTensorArenaSize];

// Variables TFLite
tflite::MicroInterpreter* interpreter = nullptr;
TfLiteTensor* input = nullptr;
TfLiteTensor* output = nullptr;

```

```
// Labels des classes
const char* labels[] = {"silence", "oui", "non", "marche", "arret"};

void setup() {
    Serial.begin(115200);

    // Charger le mod le
    const tflite::Model* model = tflite::GetModel(model_data);

    // R solveur d'op rations
    static tflite::AllOpsResolver resolver;

    // Cr er l'interpr teur
    static tflite::MicroInterpreter static_interpreter(
        model, resolver, tensor_arena, kTensorArenaSize);
    interpreter = &static_interpreter;

    // Allouer les tenseurs
    interpreter->AllocateTensors();

    // Pointeurs vers les tenseurs d'entr e/sortie
    input = interpreter->input(0);
    output = interpreter->output(0);

    Serial.println("Modele TinyML charge!");
}

void loop() {
    // Acquisition audio (MFCC features)
    float audio_features[49 * 40];  // 49 frames x 40 MFCC
    captureAudioFeatures(audio_features);

    // Copier dans le tenseur d'entr e
    for (int i = 0; i < 49 * 40; i++) {
        input->data.int8[i] = (int8_t)(audio_features[i] / input->params.scale
                                       + input->params.zero_point);
    }

    // Inf rence
    unsigned long start = micros();
    TfLiteStatus status = interpreter->Invoke();
    unsigned long inference_time = micros() - start;

    if (status == kTfLiteOk) {
        // Trouver la classe pr dite
        int max_idx = 0;
        float max_score = -1e9;

```

```
65          for (int i = 0; i < 5; i++) {
66              float score = (output->data.int8[i] - output->params.
                    zero_point)
67                            * output->params.scale;
68              if (score > max_score) {
69                  max_score = score;
70                  max_idx = i;
71              }
72          }
73
74          Serial.printf("Prediction: %s (%.2f%%) en %lu us\n",
75                        labels[max_idx], max_score * 100,
                              inference_time);
76      }
77
78      delay(100);
79  }
```

Listing 6.1 – Détection de mots-clés avec TensorFlow Lite Micro

6.3 Détection d'Anomalies dans les Données IoT

6.3.1 Approches de Détection

La détection d'anomalies est cruciale pour la maintenance prédictive et la sécurité des systèmes IoT.

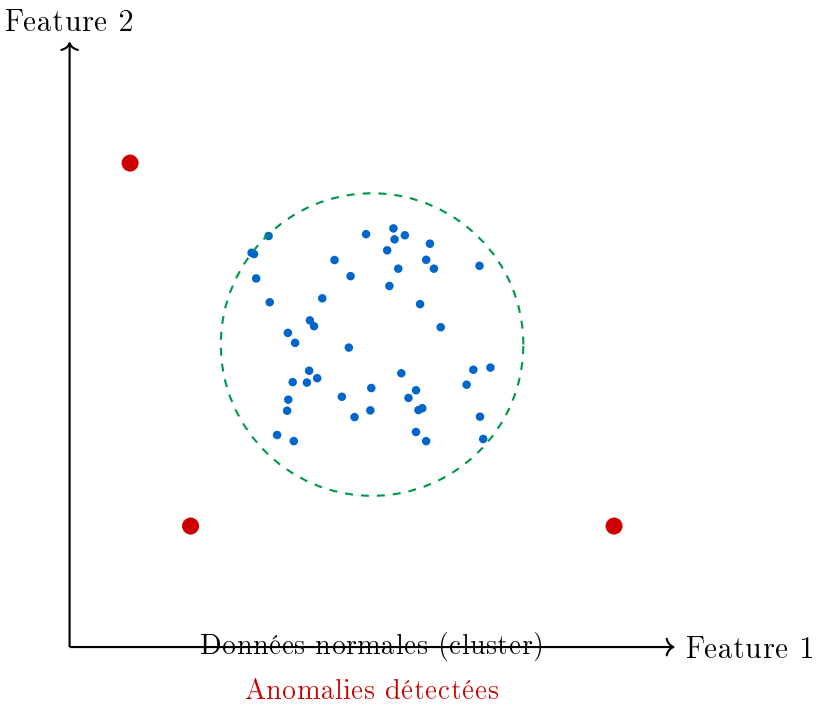


FIGURE 6.3 – Détection d'anomalies par clustering

6.3.2 Algorithmes de Détection d'Anomalies

TABLE 6.4 – Algorithmes de détection d'anomalies pour l'IoT

Algorithme	Type	Complexité	Cas d'usage IoT
Isolation Forest	Non supervisé	O(n log n)	Données tabulaires
One-Class SVM	Semi-supervisé	$O(n^2)$ à $O(n^3)$	Petit dataset
Autoencoders	Deep Learning	Variable	Séries temporelles
LSTM-AE	Deep Learning	Élevée	Séquences longues
Z-score	Statistique	O(n)	Temps réel simple
DBSCAN	Clustering	O(n log n)	Clustering spatial

6.3.3 Exemple : Autoencoder pour Détection d'Anomalies

```python
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, LSTM,
    RepeatVector, TimeDistributed

class IoTAnomalyDetector:
    def __init__(self, sequence_length=60, n_features=5):
        self.sequence_length = sequence_length
        self.n_features = n_features
        self.model = self._build_model()
        self.threshold = None

    def _build_model(self):
        # Entr e
        inputs = Input(shape=(self.sequence_length, self.n_features
            ))

        # Encoder
        x = LSTM(64, activation='relu', return_sequences=True)(
            inputs)
        x = LSTM(32, activation='relu', return_sequences=False)(x)

        # Latent space
        encoded = Dense(16, activation='relu')(x)

        # Decoder
        x = RepeatVector(self.sequence_length)(encoded)
        x = LSTM(32, activation='relu', return_sequences=True)(x)
        x = LSTM(64, activation='relu', return_sequences=True)(x)

        # Sortie
        outputs = TimeDistributed(Dense(self.n_features))(x)

        model = Model(inputs, outputs)
        model.compile(optimizer='adam', loss='mse')
        return model
```

```
35
36      def train(self, normal_data, epochs=50, batch_size=32):
37          """Entra ner sur donn es normales uniquement"""
38          history = self.model.fit(
39              normal_data, normal_data,
40              epochs=epochs,
41              batch_size=batch_size,
42              validation_split=0.1,
43              verbose=1
44          )
45
46          # Calculer le seuil (99e percentile des erreurs)
47          reconstructions = self.model.predict(normal_data)
48          mse = np.mean(np.power(normal_data - reconstructions, 2),
                axis=(1,2))
49          self.threshold = np.percentile(mse, 99)
50
51          return history
52
53      def detect_anomaly(self, data):
54          """D tecter les anomalies"""
55          reconstruction = self.model.predict(data)
56          mse = np.mean(np.power(data - reconstruction, 2), axis
                =(1,2))
57          return mse > self.threshold, mse
58
59  # Utilisation
60  detector = IoTAnomalyDetector(sequence_length=60, n_features=5)
61
62  # Entra nement sur donn es normales
63  detector.train(normal_sensor_data, epochs=100)
64
65  # D tection en temps r el
66  def process_iot_stream(new_data):
67      is_anomaly, score = detector.detect_anomaly(new_data)
68      if is_anomaly:
69          send_alert(f"Anomalie␣detectee!␣Score:␣{score:.4f}")
```

Listing 6.2 – Autoencoder pour détection d'anomalies IoT

6.4 Maintenance Prédictive

6.4.1 Principe

La **maintenance prédictive** utilise les données des capteurs IoT et le machine learning pour prédire les défaillances avant qu'elles ne surviennent, permettant une intervention proactive.

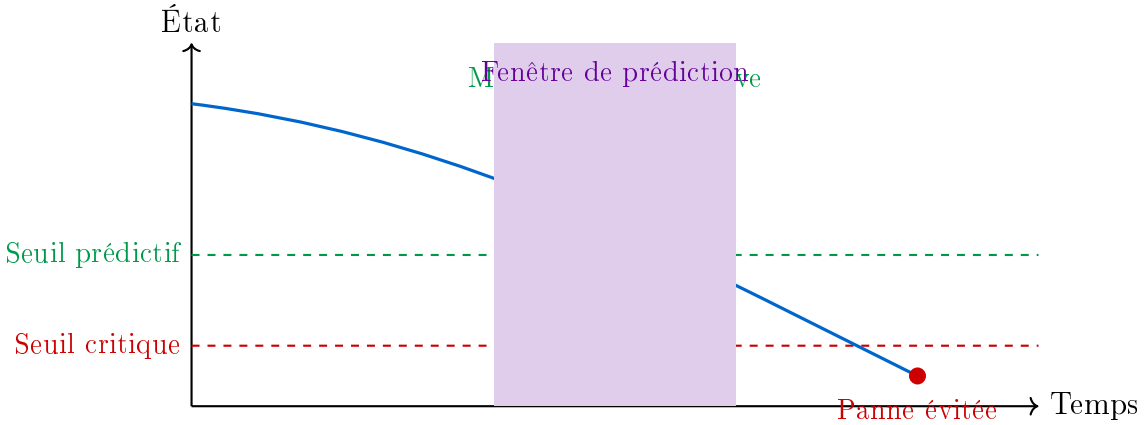


FIGURE 6.4 – Principe de la maintenance prédictive

6.4.2 Indicateurs de Santé des Équipements

TABLE 6.5 – Indicateurs de santé couramment surveillés

Équipement	Capteurs	Indicateurs	Défauts détectables
Moteur électrique	Vibration, courant, température	RMS, kurtosis, spectre FFT	Déséquilibre, roulement, bobinage
Pompe	Pression, débit, vibration	Cavitation index, rendement	Cavitation, fuite, usure
Compresseur	Pression, température, courant	Ratio compression, consommation	Valve, joint, surcharge
Roulement	Vibration, température acoustique	BPFO, BPFI, BSF, FTF	Cage, piste int/ext, bille

6.4.3 Modèle de Prédiction de RUL

La **Remaining Useful Life** (RUL) représente le temps restant avant défaillance :

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Dropout

def build_rul_model(sequence_length, n_features):
    """
    Mod le LSTM pour pr diction de Remaining Useful Life
    """
    model = Sequential([
        LSTM(128, input_shape=(sequence_length, n_features),
             return_sequences=True),
        Dropout(0.2),
        LSTM(64, return_sequences=True),
        Dropout(0.2),
        LSTM(32, return_sequences=False),
        Dropout(0.2),
        Dense(32, activation='relu'),
```

```
        Dense(1, activation='linear')  # RUL en cycles/heures
    ])

    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
        loss='mse',
        metrics=['mae']
    )

    return model

# Entra nement
model = build_rul_model(sequence_length=50, n_features=21)
model.fit(X_train, y_train_rul,
          validation_data=(X_val, y_val_rul),
          epochs=100, batch_size=256)

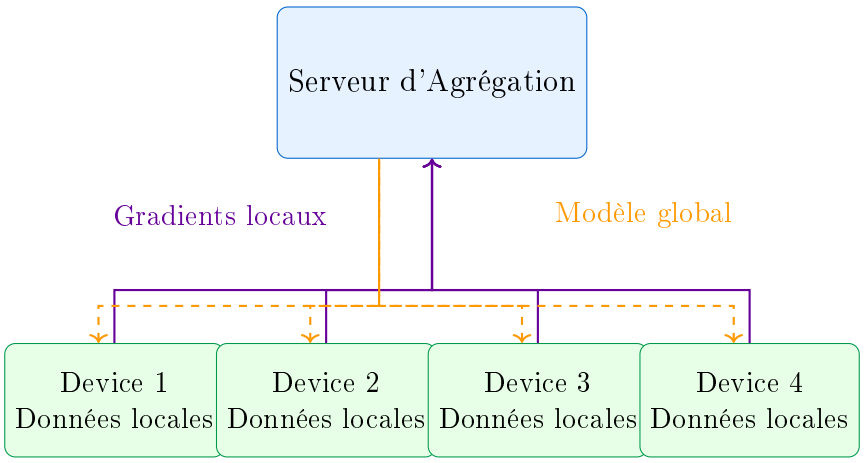
# Pr diction
predicted_rul = model.predict(current_sensor_window)
print(f"RUL predit: {predicted_rul[0][0]:.1f} cycles")
```

Listing 6.3 – Modèle LSTM pour prédiction de RUL

6.5 Federated Learning pour l'IoT

6.5.1 Principe

Le **Federated Learning** (apprentissage fédéré) permet d'entraîner des modèles ML de manière distribuée sur les dispositifs IoT, sans centraliser les données brutes.



Les données restent sur les dispositifs. Seuls les gradients/poids sont partagés.

FIGURE 6.5 – Architecture de Federated Learning

6.5.2 Avantages pour l'IoT

1. **Confidentialité** : les données sensibles ne quittent pas le dispositif

2. **Bande passante** : réduction drastique des transferts de données
3. **Latence** : entraînement et inférence locaux
4. **Robustesse** : tolérance aux pannes et déconnexions
5. **Personnalisation** : modèles adaptés au contexte local

6.5.3 Algorithme FedAvg

L'algorithme **Federated Averaging** (FedAvg) est le plus répandu :

Algorithm 1 Federated Averaging (FedAvg)

Initialisation : Modèle global w_0
for chaque round $t = 1, 2, ..., T$ **do**
  Sélectionner un sous-ensemble S_t de K dispositifs
  for chaque dispositif $k \in S_t$ **en parallèle do**
    $w_k^{t+1} \leftarrow \text{LocalUpdate}(k, w^t)$
  end for
  $w^{t+1} \leftarrow \sum_{k \in S_t} \frac{n_k}{n} w_k^{t+1}$ ▷ Agrégation pondérée
end for
procedure LocalUpdate(k, w)
  $\mathcal{B} \leftarrow$ diviser données locales en batches
  for chaque epoch local $e = 1, ..., E$ **do**
    for chaque batch $b \in \mathcal{B}$ **do**
      $w \leftarrow w - \eta \nabla \mathcal{L}(w; b)$
    end for
  end for
  return w
end procedure

Chapitre 7

Industrie 4.0 et Internet Industriel des Objets (IIoT)

7.1 Introduction à l'Industrie 4.0

7.1.1 Définition

L'**Industrie 4.0**, ou quatrième révolution industrielle, désigne la transformation numérique de l'industrie manufacturière par l'intégration de technologies avancées : IoT, intelligence artificielle, robotique, Big Data, et systèmes cyber-physiques.

7.1.2 Les Neuf Piliers Technologiques

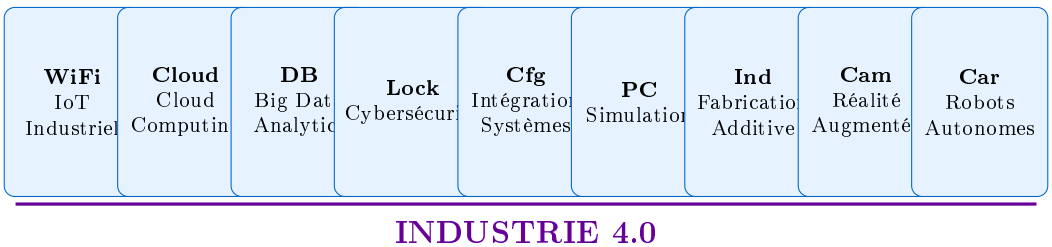


FIGURE 7.1 – Les neuf piliers technologiques de l'Industrie 4.0

7.2 Internet Industriel des Objets (IIoT)

7.2.1 Différences entre IoT et IIoT

TABLE 7.1 – Comparaison IoT grand public vs IIoT

Critère	IoT Grand Public	IIoT
Environnement	Domicile, ville	Usine, infrastructure critique
Criticité	Confort	Sécurité, production
Fiabilité requise	99%	99.999% (5 nines)
Latence	Secondes tolérées	Millisecondes critiques
Durée de vie	2-5 ans	10-30 ans
Interopérabilité	Variable	Standards industriels stricts
Sécurité	Important	Critique (safety + security)
Volume données	Ko-Mo/jour	Go-To/jour

7.2.2 Architecture de Référence IIoT

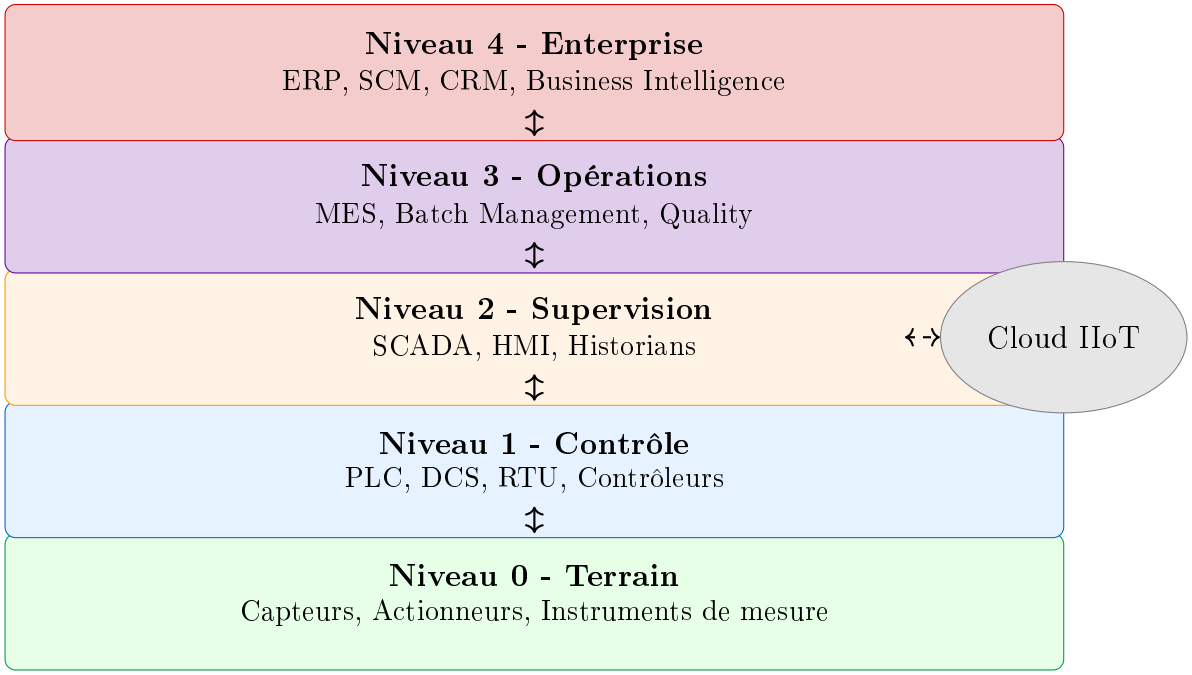


FIGURE 7.2 – Modèle de référence Purdue/ISA-95 étendu

7.3 Protocoles Industriels

7.3.1 OPC UA (Unified Architecture)

OPC UA est le standard d'interopérabilité pour l'industrie 4.0, permettant l'échange sécurisé de données entre équipements hétérogènes.

TABLE 7.2 – Caractéristiques OPC UA

Caractéristique	Description
Architecture	Client-Serveur, Pub/Sub
Transport	TCP, WebSocket, MQTT, AMQP
Sécurité	Chiffrement, authentification X.509, signatures
Modèle données	Orienté objet, hiérarchique, typage fort
Découverte	Discovery Server, mDNS
Historisation	Accès aux données historiques natif
Companion Specs	Robotics, CNC, PackML, Plastics, etc.

7.3.2 Modbus et ses Évolutions

Modbus reste très utilisé dans l'industrie malgré son âge (1979) :

TABLE 7.3 – Variantes du protocole Modbus

Variante	Couche physique	Vitesse max	Distance	Sécurité
Modbus RTU	RS-485/RS-232	115.2 kbps	1200 m	Aucune
Modbus ASCII	RS-485/RS-232	19.2 kbps	1200 m	Aucune
Modbus TCP	Ethernet	100+ Mbps	LAN/WAN	TLS optionnel
Modbus UDP	Ethernet	100+ Mbps	LAN	Aucune

7.3.3 Protocoles Temps Réel

TABLE 7.4 – Protocoles industriels temps réel

Protocole	Latence	Jitter	Débit	Domaine
EtherCAT	$<$100 μs	$<$1 μs	100 Mbps	Motion control
PROFINET IRT	$<$1 ms	$<$1 μs	100 Mbps	Automatisation
EtherNet/IP CIP Sync	$<$1 ms	$<$100 μs	1 Gbps	Process control
POWERLINK	$<$400 μs	$<$1 μs	100 Mbps	Motion control
TSN (IEEE 802.1)	$<$2 ms	Déterministe	1-100 Gbps	Convergence IT/OT

7.4 Jumeaux Numériques (Digital Twins)

7.4.1 Concept

Un **jumeau numérique** est une réplique virtuelle d'un équipement, processus ou système physique, alimentée en temps réel par les données des capteurs IoT.

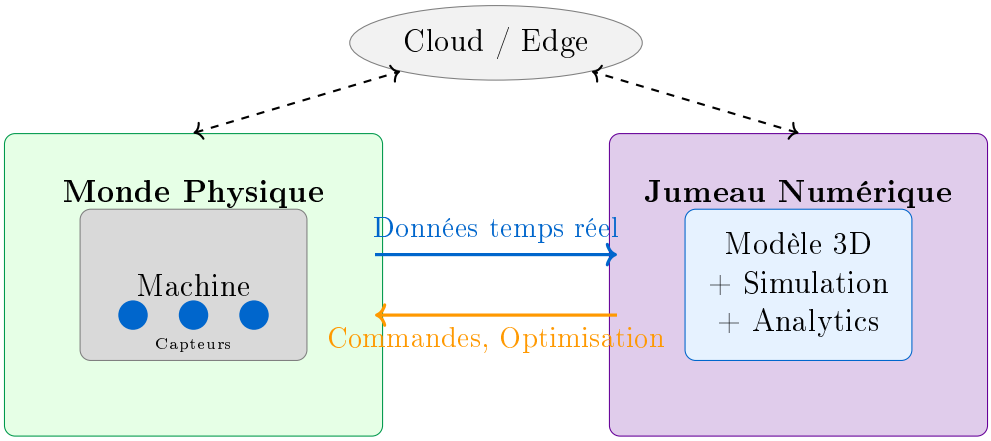


FIGURE 7.3 – Architecture de jumeau numérique

7.4.2 Applications des Jumeaux Numériques

1. **Conception** : simulation et optimisation avant fabrication
2. **Commissioning virtuel** : test des automatismes sur le jumeau
3. **Monitoring** : visualisation temps réel de l'état
4. **Prédiction** : simulation de scénarios what-if
5. **Maintenance** : diagnostic et planification d'interventions
6. **Formation** : entraînement des opérateurs sans risque

7.5 Cas d'Usage IIoT

7.5.1 Monitoring de Ligne de Production

```python
from opcua import Client
import paho.mqtt.client as mqtt
import json
import time

class ProductionLineMonitor:
    def __init__(self, opc_url, mqtt_broker):
        self.opc_client = Client(opc_url)
        self.mqtt_client = mqtt.Client()
        self.mqtt_broker = mqtt_broker

    def connect(self):
        self.opc_client.connect()
        self.mqtt_client.connect(self.mqtt_broker)

    def read_production_data(self):
        """Lire les KPIs de la ligne de production"""
        root = self.opc_client.get_root_node()

        # Parcourir les noeuds OPC UA
```

```
        data = {
            "timestamp": time.time(),
            "line_id": "LINE_01",
            "machines": []
        }

        # Exemple : 5 machines sur la ligne
        for i in range(1, 6):
            machine_node = root.get_child([
                "0:Objects", f"2:Machine_{i}"
            ])

            machine_data = {
                "id": f"Machine_{i}",
                "state": machine_node.get_child("2:State").
                    get_value(),
                "speed_rpm": machine_node.get_child("2:Speed").
                    get_value(),
                "temperature": machine_node.get_child("2:
                    Temperature").get_value(),
                "vibration": machine_node.get_child("2:Vibration").
                    get_value(),
                "parts_count": machine_node.get_child("2:PartsCount
                    ").get_value(),
                "oee": machine_node.get_child("2:OEE").get_value()
            }
            data["machines"].append(machine_data)

        return data

    def calculate_line_oee(self, data):
        """Calculer l'OEE global de la ligne"""
        availability = sum(1 for m in data["machines"]
                           if m["state"] == "RUNNING") / len(data["
                              machines"])
        avg_oee = sum(m["oee"] for m in data["machines"]) / len(
            data["machines"])
        return availability, avg_oee

    def publish_to_cloud(self, data):
        """Publier les donn es vers le cloud via MQTT"""
        topic = f"factory/production/{data['line_id']}"
        self.mqtt_client.publish(topic, json.dumps(data), qos=1)

    def run(self, interval=5):
        """Boucle principale de monitoring"""
        self.connect()

        while True:
            data = self.read_production_data()
            availability, oee = self.calculate_line_oee(data)
```

```
65
66              data["line_availability"] = availability
67              data["line_oee"] = oee
68
69              self.publish_to_cloud(data)
70
71              print(f"Line␣OEE:␣{oee:.1%},␣Availability:␣{
                    availability:.1%}")
72              time.sleep(interval)
73
74  # Utilisation
75  monitor = ProductionLineMonitor(
76      opc_url="opc.tcp://plc-server:4840",
77      mqtt_broker="mqtt.factory.local"
78  )
79  monitor.run()
```

Listing 7.1 – Collecte de données OPC UA depuis une ligne de production

Chapitre 8

Ville Intelligente (Smart City)

8.1 Concept et Enjeux

8.1.1 Définition

Une **ville intelligente** (Smart City) utilise les technologies numériques et l'IoT pour améliorer la qualité de vie des citoyens, optimiser les services urbains et réduire l'empreinte environnementale.

8.1.2 Les Six Dimensions de la Smart City

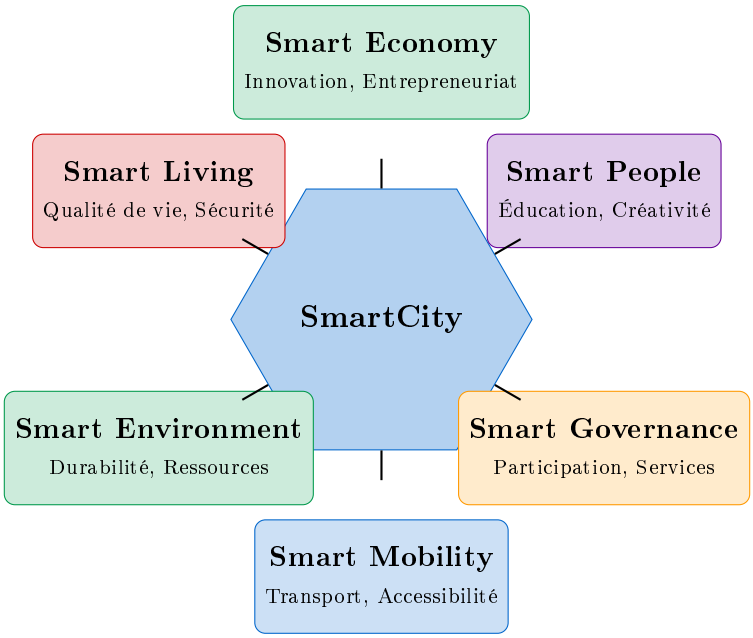


FIGURE 8.1 – Les six dimensions de la Smart City

8.2 Infrastructure IoT Urbaine

8.2.1 Architecture Globale

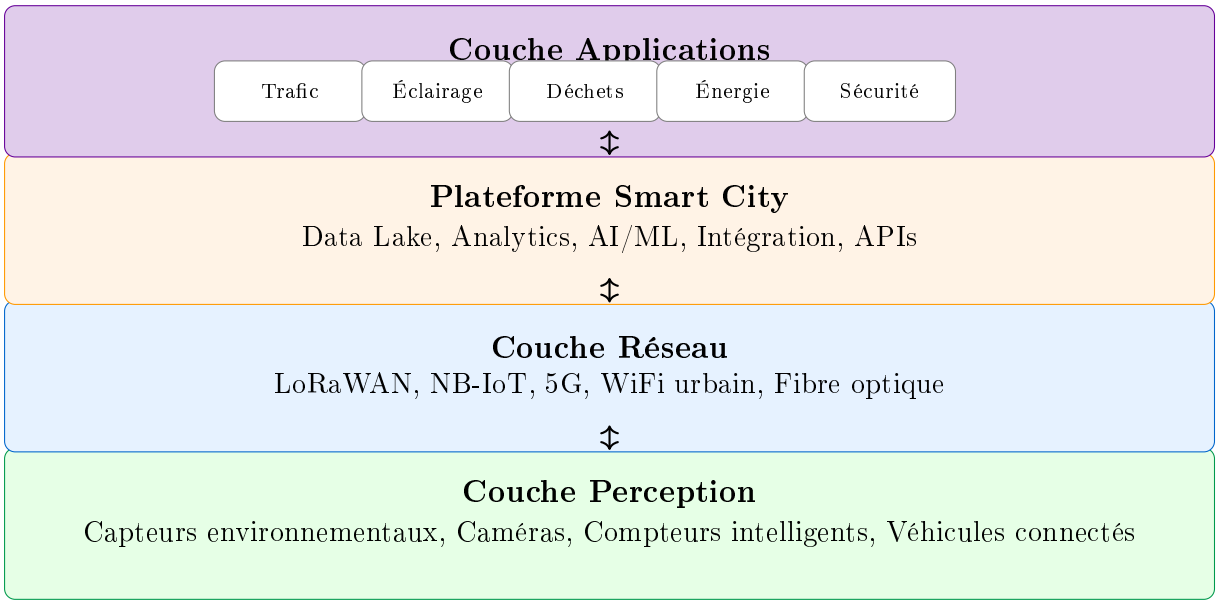


FIGURE 8.2 – Architecture IoT d'une Smart City

8.3 Gestion Intelligente du Trafic

8.3.1 Composants du Système

— **Capteurs de comptage** : boucles magnétiques, radar, caméras
— **Feux adaptatifs** : ajustement en temps réel selon le flux
— **Panneaux à message variable** : information des usagers
— **V2X** (Vehicle-to-Everything) : communication véhicule-infrastructure
— **Centrale de régulation** : coordination globale

8.3.2 Algorithmes d'Optimisation

```python
import numpy as np
from collections import deque
import random

class TrafficLightAgent:
    """Agent RL pour optimisation d'un carrefour"""

    def __init__(self, state_size, action_size):
        self.state_size = state_size  # [queue_N, queue_S, queue_E,
            queue_W, current_phase, time_in_phase]
        self.action_size = action_size  # Phases possibles
        self.memory = deque(maxlen=10000)
```

```
        self.gamma = 0.95  # Discount factor
        self.epsilon = 1.0  # Exploration rate
        self.epsilon_min = 0.01
        self.epsilon_decay = 0.995
        self.learning_rate = 0.001
        self.model = self._build_model()

    def _build_model(self):
        """R seau de neurones Q-learning"""
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.layers import Dense

        model = Sequential([
            Dense(64, input_dim=self.state_size, activation='relu'),
            Dense(64, activation='relu'),
            Dense(32, activation='relu'),
            Dense(self.action_size, activation='linear')
        ])
        model.compile(loss='mse', optimizer='adam')
        return model

    def get_state(self, intersection_data):
        """Extraire l' tat  depuis les capteurs"""
        return np.array([
            intersection_data['queue_north'],
            intersection_data['queue_south'],
            intersection_data['queue_east'],
            intersection_data['queue_west'],
            intersection_data['current_phase'],
            intersection_data['time_in_phase']
        ])

    def choose_action(self, state):
        """Epsilon-greedy action selection"""
        if np.random.rand() <= self.epsilon:
            return random.randrange(self.action_size)
        q_values = self.model.predict(state.reshape(1, -1), verbose=0)
        return np.argmax(q_values[0])

    def calculate_reward(self, prev_state, action, new_state):
        """R compense bas e sur r duction des files d'attente"""
        prev_total_queue = sum(prev_state[:4])
        new_total_queue = sum(new_state[:4])

        # R compense = r duction de la file totale
        reward = prev_total_queue - new_total_queue

        # P nalit  pour changement de phase trop fr quent
        if prev_state[4] != action and prev_state[5] < 10:
```

```
61              reward -= 5
62
63          return reward
64
65      def train(self, batch_size=32):
66          """Entra nement par experience replay"""
67          if len(self.memory) < batch_size:
68              return
69
70          minibatch = random.sample(self.memory, batch_size)
71
72          for state, action, reward, next_state, done in minibatch:
73              target = reward
74              if not done:
75                  target += self.gamma * np.amax(
76                      self.model.predict(next_state.reshape(1, -1),
                            verbose=0)[0]
77                  )
78
79              target_f = self.model.predict(state.reshape(1, -1),
                    verbose=0)
80              target_f[0][action] = target
81
82              self.model.fit(state.reshape(1, -1), target_f, epochs
                    =1, verbose=0)
83
84          if self.epsilon > self.epsilon_min:
85              self.epsilon *= self.epsilon_decay
```

Listing 8.1 – Optimisation des feux de circulation par apprentissage par renforcement

8.4 Éclairage Public Intelligent

8.4.1 Architecture du Système

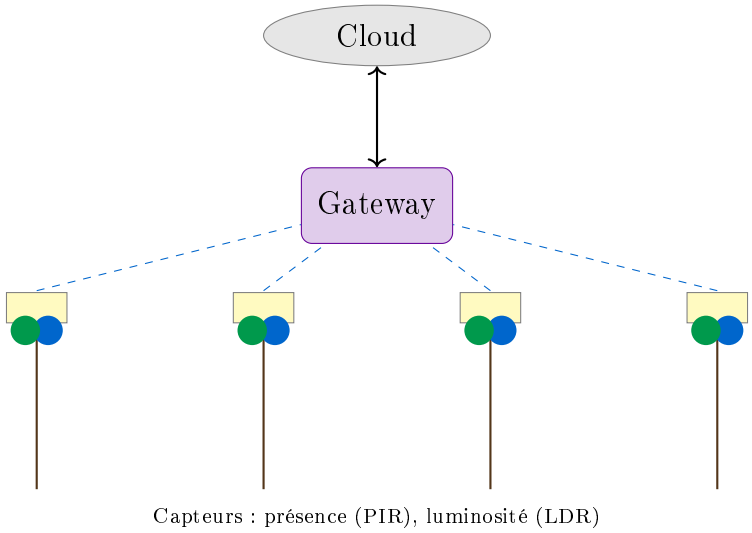


FIGURE 8.3 – Architecture d'éclairage intelligent

8.4.2 Logique de Contrôle

```
#include <LoRa.h>
#include <ArduinoJson.h>

// Pins
#define PIR_PIN 2
#define LDR_PIN A0
#define LED_PWM_PIN 9

// Configuration
#define LUX_THRESHOLD_ON 50       // Lux pour allumage
#define LUX_THRESHOLD_OFF 100     // Lux pour extinction
#define PRESENCE_BOOST 100        // % en cas de presence
#define BASE_BRIGHTNESS 30        // % en mode eco
#define TIMEOUT_PRESENCE 60000    // 60s apres passage

// Variables
unsigned long lastPresence = 0;
int currentBrightness = 0;
bool nightMode = false;

struct LampStatus {
    uint8_t brightness;
    uint16_t lux;
    bool presence;
    float energy_kwh;
    uint32_t uptime;
```

```
} status;

void setup() {
    Serial.begin(115200);
    pinMode(PIR_PIN, INPUT);
    pinMode(LED_PWM_PIN, OUTPUT);

    // Init LoRa
    if (!LoRa.begin(868E6)) {
        Serial.println("LoRa init failed!");
        while (1);
    }

    Serial.println("Smart Streetlight initialized");
}

int readLux() {
    int raw = analogRead(LDR_PIN);
    // Conversion approximative en lux
    return map(raw, 0, 1023, 0, 1000);
}

bool detectPresence() {
    return digitalRead(PIR_PIN) == HIGH;
}

int calculateBrightness(int lux, bool presence) {
    // Jour : eteint
    if (lux > LUX_THRESHOLD_OFF) {
        return 0;
    }

    // Nuit
    if (lux < LUX_THRESHOLD_ON) {
        if (presence) {
            return PRESENCE_BOOST;  // Pleine puissance
        } else if (millis() - lastPresence < TIMEOUT_PRESENCE) {
            // Diminution progressive apres passage
            int elapsed = millis() - lastPresence;
            int fade = map(elapsed, 0, TIMEOUT_PRESENCE,
                           PRESENCE_BOOST, BASE_BRIGHTNESS);
            return fade;
        } else {
            return BASE_BRIGHTNESS;  // Mode eco
        }
    }

    // Transition crepuscule
    return map(lux, LUX_THRESHOLD_ON, LUX_THRESHOLD_OFF,
               BASE_BRIGHTNESS, 0);
}
```

```
78
79  void setLED(int brightness) {
80      int pwm = map(brightness, 0, 100, 0, 255);
81      analogWrite(LED_PWM_PIN, pwm);
82      currentBrightness = brightness;
83  }
84
85  void sendStatus() {
86      StaticJsonDocument<200> doc;
87      doc["id"] = "LAMP_001";
88      doc["brightness"] = status.brightness;
89      doc["lux"] = status.lux;
90      doc["presence"] = status.presence;
91      doc["energy"] = status.energy_kwh;
92
93      char buffer[200];
94      serializeJson(doc, buffer);
95
96      LoRa.beginPacket();
97      LoRa.print(buffer);
98      LoRa.endPacket();
99  }
100
101 void loop() {
102     // Lecture capteurs
103     int lux = readLux();
104     bool presence = detectPresence();
105
106     if (presence) {
107         lastPresence = millis();
108     }
109
110     // Calcul luminosite
111     int targetBrightness = calculateBrightness(lux, presence);
112
113     // Transition douce
114     if (targetBrightness > currentBrightness) {
115         setLED(currentBrightness + 1);
116     } else if (targetBrightness < currentBrightness) {
117         setLED(currentBrightness - 1);
118     }
119
120     // Mise a jour status
121     status.brightness = currentBrightness;
122     status.lux = lux;
123     status.presence = presence;
124
125     // Envoi periodique (toutes les 5 min)
126     static unsigned long lastSend = 0;
127     if (millis() - lastSend > 300000) {
128         sendStatus();
```

```
129             lastSend = millis();
130         }
131
132         delay(100);
133     }
```

Listing 8.2 – Contrôle adaptatif d'un lampadaire LED

8.5 Gestion Intelligente des Déchets

8.5.1 Système de Poubelles Connectées

Les poubelles intelligentes intègrent des capteurs de niveau et communiquent leur état pour optimiser les tournées de collecte.

TABLE 8.1 – ROI d'un système de gestion intelligente des déchets

Indicateur	**Avant IoT**	**Après IoT**
Fréquence de collecte	Fixe (quotidienne)	À la demande
Taux de remplissage moyen	40%	85%
Kilomètres parcourus	100%	-30%
Coût carburant	100%	-25%
Débordements	Fréquents	Rares (<2%)
Coût opérationnel	100%	-20 à -40%

Chapitre 9

Santé Connectée et Internet des Objets Médicaux (IoMT)

9.1 Introduction à l'IoT Médical

9.1.1 Définition et Enjeux

L'**Internet des Objets Médicaux** (IoMT - Internet of Medical Things) désigne l'ensemble des dispositifs médicaux connectés permettant la collecte, la transmission et l'analyse de données de santé en temps réel.

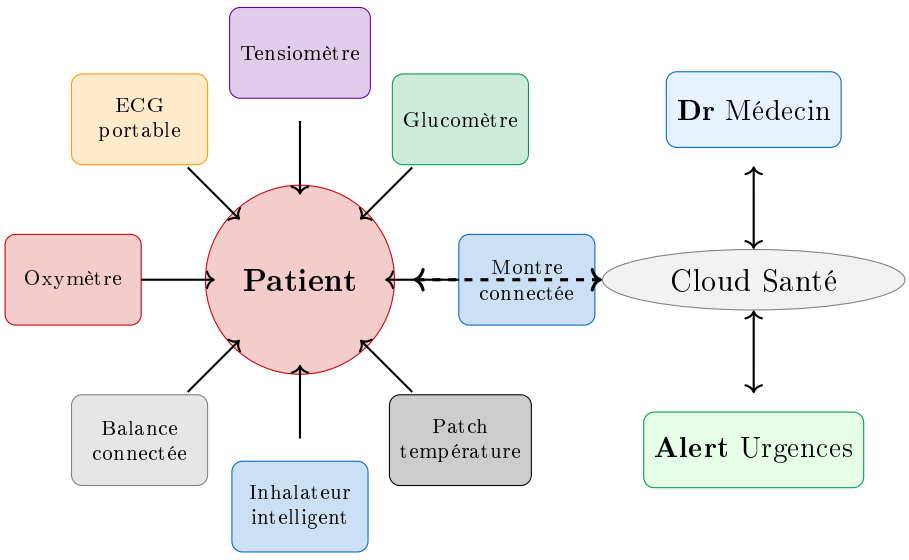


FIGURE 9.1 – Écosystème IoT de santé connectée

9.1.2 Marché et Perspectives

TABLE 9.1 – Évolution du marché IoMT mondial

Indicateur	2020	2023	2025	2030
Marché mondial (Mds $)	41	89	158	446
Dispositifs connectés (millions)	161	320	500	950+
Données générées (Eo/an)	2,3	5,2	10,1	35+

9.2 Dispositifs Médicaux Connectés

9.2.1 Classification des Dispositifs IoMT

TABLE 9.2 – Classification des dispositifs IoMT

Catégorie	Exemples	Paramètres	Protocoles
Wearables	Montres, bracelets	FC, SpO2, activité	BLE, WiFi
Dispositifs portables	Glucomètres, ECG	Glycémie, rythme	BLE, NFC
Implants	Pacemakers, pompes	Stimulation, débit	Propriétaire, MICS
Équipements cliniques	Moniteurs, imagerie	Multi-paramètres	WiFi, Ethernet
Environnement	Capteurs chambre	T°, qualité air	ZigBee, LoRa

9.2.2 Exemple : Système de Monitoring Cardiaque

```
#include <WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>

// Configuration ECG AD8232
#define ECG_PIN 34         // Sortie analogique
#define LO_PLUS_PIN 25     // Lead-off detection +
#define LO_MINUS_PIN 26    // Lead-off detection -

// Configuration
const int SAMPLE_RATE = 250;  // Hz (standard ECG)
const int BUFFER_SIZE = 250;  // 1 seconde de donnees
float ecgBuffer[BUFFER_SIZE];
int bufferIndex = 0;

// Seuils de detection
float heartRateThreshold = 0.6;
unsigned long lastBeat = 0;
float heartRate = 0;

// MQTT
const char* mqtt_topic_ecg = "health/patient001/ecg";
const char* mqtt_topic_hr = "health/patient001/heartrate";
const char* mqtt_topic_alert = "health/patient001/alert";

```

```
WiFiClient espClient;
PubSubClient mqtt(espClient);

void setup() {
    Serial.begin(115200);

    pinMode(LO_PLUS_PIN, INPUT);
    pinMode(LO_MINUS_PIN, INPUT);

    // Connexion WiFi et MQTT
    setupWiFi();
    mqtt.setServer("mqtt.health.local", 8883);
    mqtt.setCallback(mqttCallback);

    // Timer pour echantillonnage precis
    timerAttachInterrupt(timer, &onTimer, true);
    timerAlarmWrite(timer, 1000000 / SAMPLE_RATE, true);
    timerAlarmEnable(timer);
}

void IRAM_ATTR onTimer() {
    // Verifier lead-off
    if (digitalRead(LO_PLUS_PIN) == 1 || digitalRead(LO_MINUS_PIN)
       == 1) {
        ecgBuffer[bufferIndex] = -1;  // Signal invalide
    } else {
        // Lecture ECG (12-bit ADC)
        int raw = analogRead(ECG_PIN);
        float voltage = (raw / 4095.0) * 3.3;
        ecgBuffer[bufferIndex] = voltage;

        // Detection de battement (algorithme simplifie)
        detectHeartbeat(voltage);
    }

    bufferIndex = (bufferIndex + 1) % BUFFER_SIZE;
}

void detectHeartbeat(float sample) {
    static float filteredSample = 0;
    static float threshold = 0.5;
    static bool beatDetected = false;

    // Filtre passe-haut simple
    filteredSample = 0.9 * filteredSample + 0.1 * sample;
    float derivative = sample - filteredSample;

    // Detection de pic R
    if (derivative > threshold && !beatDetected) {
        unsigned long now = millis();
        if (now - lastBeat > 300) {  // Periode refractaire
```

```
            float interval = (now - lastBeat) / 1000.0;
            heartRate = 60.0 / interval;
            lastBeat = now;

            // Verifier anomalies
            if (heartRate < 40 || heartRate > 180) {
                sendAlert("Frequence cardiaque anormale", heartRate);
            }
        }
        beatDetected = true;
    } else if (derivative < threshold * 0.5) {
        beatDetected = false;
    }
}

void sendAlert(const char* message, float value) {
    StaticJsonDocument<256> doc;
    doc["patient_id"] = "P001";
    doc["alert_type"] = "cardiac";
    doc["message"] = message;
    doc["value"] = value;
    doc["timestamp"] = millis();
    doc["priority"] = "HIGH";

    char buffer[256];
    serializeJson(doc, buffer);
    mqtt.publish(mqtt_topic_alert, buffer, true);  // Retained

    Serial.printf("ALERTE: %s (%.1f)\n", message, value);
}

void publishECGData() {
    // Envoyer les donnees ECG par paquets
    StaticJsonDocument<1024> doc;
    doc["patient_id"] = "P001";
    doc["sample_rate"] = SAMPLE_RATE;

    JsonArray samples = doc.createNestedArray("samples");
    for (int i = 0; i < 50; i++) {  // 50 echantillons a la fois
        int idx = (bufferIndex - 50 + i + BUFFER_SIZE) % BUFFER_SIZE;
        samples.add(ecgBuffer[idx]);
    }

    char buffer[1024];
    serializeJson(doc, buffer);
    mqtt.publish(mqtt_topic_ecg, buffer);
}

void publishHeartRate() {
```

```
    StaticJsonDocument<128> doc;
    doc["patient_id"] = "P001";
    doc["heart_rate"] = heartRate;
    doc["timestamp"] = millis();

    char buffer[128];
    serializeJson(doc, buffer);
    mqtt.publish(mqtt_topic_hr, buffer);
}

void loop() {
    mqtt.loop();

    static unsigned long lastPublish = 0;
    if (millis() - lastPublish > 200) {  // 5 Hz
        publishECGData();
        publishHeartRate();
        lastPublish = millis();
    }
}
```

Listing 9.1 – Monitoring ECG avec ESP32 et AD8232

9.3 Télémédecine et Monitoring à Distance

9.3.1 Architecture de Télésurveillance

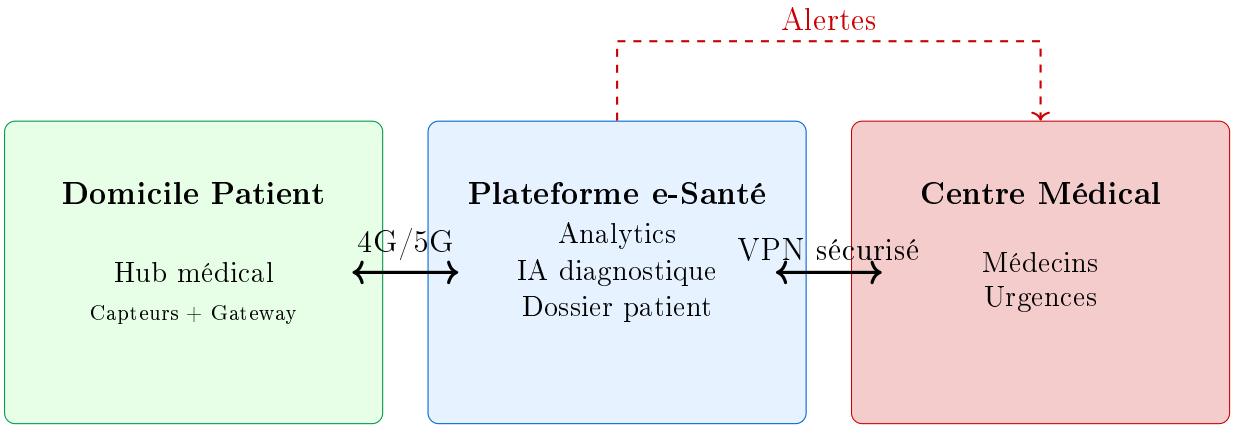


FIGURE 9.2 – Architecture de télésurveillance médicale

9.3.2 Cas d'Usage : Suivi du Diabète

1. **Mesure en continu** : Capteur de glucose sous-cutané (CGM) mesurant toutes les 5 minutes
2. **Transmission** : Données envoyées au smartphone via BLE
3. **Analyse** : Algorithme de prédiction des hypoglycémies

4. **Alerte** : Notification patient et équipe soignante si seuils dépassés
5. **Ajustement** : Pompe à insuline connectée ajustant automatiquement le débit

9.4 Réglementation et Sécurité des Données de Santé

9.4.1 Cadre Réglementaire

TABLE 9.3 – Principales réglementations IoMT

Région	Réglementation	Exigences clés
Europe	RGPD	Consentement, droit à l'oubli, DPO
	MDR 2017/745	Certification CE, évaluation clinique
	HDS	Hébergement données de santé agréé
USA	HIPAA	Confidentialité, audit trail
	FDA 21 CFR Part 820	QMS, validation logiciel
International	IEC 62443	Cybersécurité industrielle
	ISO 13485	Système qualité dispositifs médicaux

9.4.2 Sécurité Spécifique IoMT

— **Authentification forte** : certificats X.509, biométrie
— **Chiffrement bout-en-bout** : TLS 1.3, chiffrement au repos (AES-256)
— **Pseudonymisation** : dissociation identité/données médicales
— **Audit et traçabilité** : journalisation de tous les accès
— **Mise à jour sécurisée** : OTA signée, rollback
— **Résilience** : fonctionnement dégradé en cas de perte réseau

Chapitre 10

Agriculture Intelligente (Smart Farming)

10.1 Introduction à l'Agriculture de Précision

10.1.1 Définition

L'**agriculture de précision** ou **agriculture intelligente** utilise les technologies IoT, l'analyse de données et l'automatisation pour optimiser les pratiques agricoles, réduire les intrants et augmenter les rendements de manière durable.

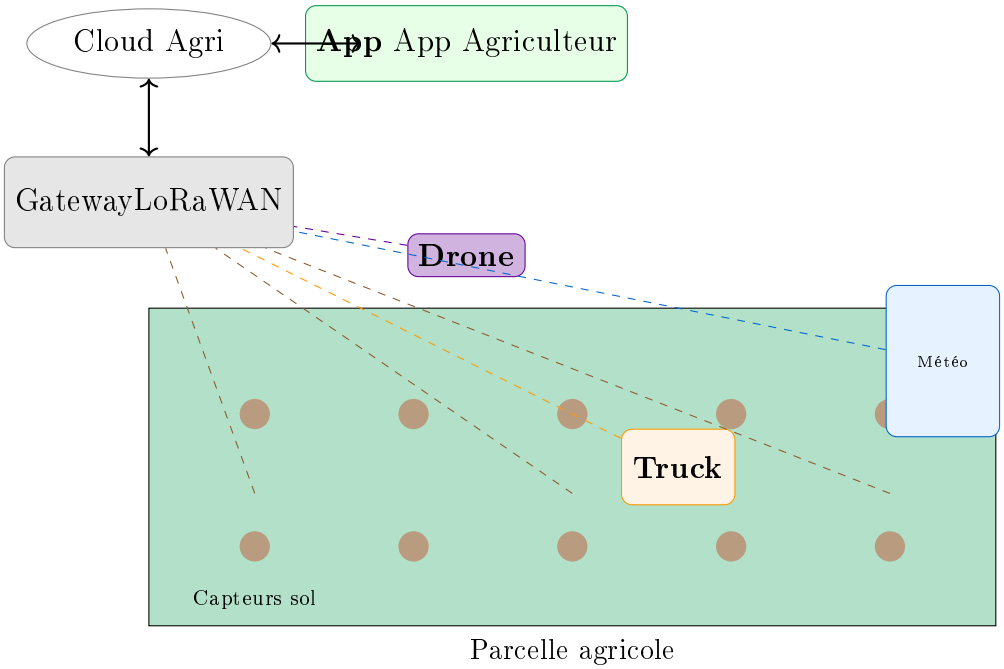


FIGURE 10.1 – Écosystème IoT agricole

10.2 Capteurs Agricoles

10.2.1 Capteurs de Sol

TABLE 10.1 – Capteurs de sol pour l'agriculture de précision

Paramètre	Capteur	Technologie	Plage	Précision
Humidité sol	Capacitif	FDR, TDR	0-100% VWC	±2%
Température sol	Thermistance	NTC	-40 à +80°C	±0.5°C
Conductivité (EC)	Électrodes	Résistif	0-20 mS/cm	±5%
pH	Électrode verre	Potentiométrique	0-14 pH	±0.1 pH
NPK	Spectroscopie	NIR, Ion-selective	Variable	±10-15%

10.2.2 Système de Monitoring des Cultures

```
#include <LoRa.h>
#include <CayenneLPP.h>
#include "LowPower.h"

// Pins capteurs
#define SOIL_MOISTURE_PIN A0
#define SOIL_TEMP_PIN A1
#define BATTERY_PIN A2
#define RAIN_PIN 2

// Configuration LoRa
#define LORA_FREQ 868E6
#define SPREADING_FACTOR 10
#define TX_POWER 14

// Intervalle de mesure (secondes)
#define MEASURE_INTERVAL 900  // 15 minutes

CayenneLPP lpp(51);
volatile bool rainDetected = false;

// Calibration capteur humidite
const int DRY_VALUE = 800;   // Valeur ADC sol sec
const int WET_VALUE = 350;   // Valeur ADC sol sature

void setup() {
    Serial.begin(9600);

    // Init LoRa
    LoRa.setPins(10, 9, 2);
    if (!LoRa.begin(LORA_FREQ)) {
        Serial.println("LoRa init failed!");
        while (1);
    }
    LoRa.setSpreadingFactor(SPREADING_FACTOR);
```

```
36      LoRa.setTxPower(TX_POWER);
37
38      // Interruption pluviometre
39      pinMode(RAIN_PIN, INPUT_PULLUP);
40      attachInterrupt(digitalPinToInterrupt(RAIN_PIN), rainISR,
            FALLING);
41
42      Serial.println("Noeud␣agricole␣initialise");
43  }
44
45  void rainISR() {
46      rainDetected = true;
47  }
48
49  float readSoilMoisture() {
50      int raw = analogRead(SOIL_MOISTURE_PIN);
51      // Conversion en pourcentage VWC
52      float moisture = map(raw, DRY_VALUE, WET_VALUE, 0, 100);
53      return constrain(moisture, 0, 100);
54  }
55
56  float readSoilTemperature() {
57      int raw = analogRead(SOIL_TEMP_PIN);
58      // Conversion NTC (Steinhart-Hart simplifie)
59      float resistance = 10000.0 * (1023.0 / raw - 1.0);
60      float tempK = 1.0 / (log(resistance / 10000.0) / 3950.0 + 1.0 /
            298.15);
61      return tempK - 273.15;
62  }
63
64  float readBatteryVoltage() {
65      int raw = analogRead(BATTERY_PIN);
66      // Diviseur de tension 2:1
67      return (raw / 1023.0) * 3.3 * 2.0;
68  }
69
70  void sendData() {
71      lpp.reset();
72
73      // Canal 1: Humidite sol (en %)
74      float moisture = readSoilMoisture();
75      lpp.addRelativeHumidity(1, moisture);
76
77      // Canal 2: Temperature sol
78      float soilTemp = readSoilTemperature();
79      lpp.addTemperature(2, soilTemp);
80
81      // Canal 3: Tension batterie
82      float battery = readBatteryVoltage();
83      lpp.addAnalogInput(3, battery);
84
```

```
    // Canal 4: Detection pluie
    if (rainDetected) {
        lpp.addDigitalInput(4, 1);
        rainDetected = false;
    }

    // Envoi LoRa
    LoRa.beginPacket();
    LoRa.write(lpp.getBuffer(), lpp.getSize());
    LoRa.endPacket();

    Serial.printf("Envoye - Humidite: %.1f%%, Temp: %.1fC, Batt: %.2fV\n",
                  moisture, soilTemp, battery);
}

void loop() {
    sendData();

    // Mode basse consommation
    LoRa.sleep();
    for (int i = 0; i < MEASURE_INTERVAL / 8; i++) {
        LowPower.powerDown(SLEEP_8S, ADC_OFF, BOD_OFF);
    }
    LoRa.idle();
}
```

Listing 10.1 – Nœud capteur agricole LoRaWAN

10.3 Irrigation Intelligente

10.3.1 Algorithme de Décision d'Irrigation

```
import numpy as np
from datetime import datetime, timedelta

class SmartIrrigationController:
    def __init__(self, field_config):
        self.field_area = field_config['area_ha']
        self.crop_type = field_config['crop']
        self.soil_type = field_config['soil_type']
        self.irrigation_efficiency = field_config.get('efficiency',
            0.85)

        # Coefficients culturaux (Kc) par stade
        self.kc_values = {
            'wheat': {'initial': 0.3, 'mid': 1.15, 'late': 0.4},
            'maize': {'initial': 0.3, 'mid': 1.2, 'late': 0.6},
            'tomato': {'initial': 0.6, 'mid': 1.15, 'late': 0.8}
        }
```

```

        # Capacite au champ et point de fletrissement
        self.soil_params = {
            'sandy': {'fc': 0.12, 'wp': 0.05, 'ksat': 200},
            'loam': {'fc': 0.27, 'wp': 0.12, 'ksat': 25},
            'clay': {'fc': 0.40, 'wp': 0.20, 'ksat': 5}
        }

    def calculate_eto(self, weather_data):
        """
        Calcul de l'evapotranspiration de reference (ET0)
        Methode FAO Penman-Monteith simplifiee
        """
        T = weather_data['temperature']
        RH = weather_data['humidity']
        u2 = weather_data['wind_speed']
        Rs = weather_data['solar_radiation']

        # Pression de vapeur saturante
        es = 0.6108 * np.exp(17.27 * T / (T + 237.3))
        ea = es * RH / 100

        # Rayonnement net (approximation)
        Rn = 0.77 * Rs - 0.1

        # ET0 (mm/jour)
        delta = 4098 * es / (T + 237.3)**2
        gamma = 0.065  # Constante psychrometrique

        ET0 = (0.408 * delta * Rn + gamma * (900 / (T + 273)) * u2
           * (es - ea)) / \
              (delta + gamma * (1 + 0.34 * u2))

        return max(0, ET0)

    def get_crop_coefficient(self, days_after_planting):
        """Coefficient cultural selon le stade phenologique"""
        kc = self.kc_values.get(self.crop_type, {'initial': 0.5, '
           mid': 1.0, 'late': 0.7})

        if days_after_planting < 30:
            return kc['initial']
        elif days_after_planting < 90:
            # Interpolation lineaire
            progress = (days_after_planting - 30) / 60
            return kc['initial'] + progress * (kc['mid'] - kc['
               initial'])
        elif days_after_planting < 120:
            return kc['mid']
        else:
            return kc['late']
```

```
65
66      def calculate_water_balance(self, soil_moisture, rainfall, etc):
67          """Bilan hydrique de la parcelle"""
68          soil = self.soil_params[self.soil_type]
69
70          # Reserve facilement utilisable (RFU)
71          rfu = (soil['fc'] - soil['wp']) * 0.5  # 50% de la RU
72
73          # Deficit hydrique
74          current_deficit = soil['fc'] - soil_moisture
75
76          # Besoin en eau = ETc - pluie efficace
77          effective_rain = rainfall * 0.8  # 80% efficace
78          water_need = etc - effective_rain
79
80          return {
81              'deficit': current_deficit,
82              'water_need': max(0, water_need),
83              'rfu_depleted': soil_moisture < (soil['wp'] + rfu)
84          }
85
86      def decide_irrigation(self, sensor_data, weather_data,
            weather_forecast):
87          """
88          Decision d'irrigation basee sur:
89          - Humidite du sol actuelle
90          - Evapotranspiration
91          - Previsions meteo
92          """
93          # Calcul ET0 et ETc
94          et0 = self.calculate_eto(weather_data)
95          days_after_planting = sensor_data.get('days_after_planting'
                , 60)
96          kc = self.get_crop_coefficient(days_after_planting)
97          etc = et0 * kc
98
99          # Bilan hydrique
100         balance = self.calculate_water_balance(
101             sensor_data['soil_moisture'],
102             sensor_data.get('rainfall_24h', 0),
103             etc
104         )
105
106         # Prevision de pluie dans les prochaines 24h
107         rain_forecast = sum(f.get('precipitation', 0) for f in
                weather_forecast[:24])
108
109         decision = {
110             'irrigate': False,
111             'volume_mm': 0,
```

```
            'duration_min': 0,
            'reason': ''
        }

        # Logique de decision
        if sensor_data['soil_moisture'] < 0.15:  # Critique
            decision['irrigate'] = True
            decision['volume_mm'] = balance['deficit'] * 1000
            decision['reason'] = 'Humidite critique'

        elif balance['rfu_depleted'] and rain_forecast < 5:
            decision['irrigate'] = True
            decision['volume_mm'] = balance['water_need'] * 2  # 2
               jours d'avance
            decision['reason'] = 'RFU epuisee, pas de pluie prevue'

        elif balance['water_need'] > 5 and rain_forecast < balance[
           'water_need']:
            decision['irrigate'] = True
            decision['volume_mm'] = balance['water_need'] -
               rain_forecast
            decision['reason'] = 'Deficit hydrique'

        if decision['irrigate']:
            # Convertir en duree (debit = 10 mm/h typ.)
            flow_rate = 10  # mm/h
            decision['duration_min'] = int(decision['volume_mm'] /
               flow_rate * 60)
            decision['duration_min'] = min(decision['duration_min'
               ], 180)  # Max 3h

        return decision

# Utilisation
controller = SmartIrrigationController({
    'area_ha': 5,
    'crop': 'maize',
    'soil_type': 'loam'
})

# Donnees capteurs et meteo
sensor_data = {
    'soil_moisture': 0.18,
    'rainfall_24h': 2,
    'days_after_planting': 75
}

weather = {
    'temperature': 28,
    'humidity': 45,
    'wind_speed': 2.5,
```

```
    'solar_radiation': 22
}

decision = controller.decide_irrigation(sensor_data, weather,
    weather_forecast)
print(f"Irriguer : {decision['irrigate']}")
print(f"Volume : {decision['volume_mm']:.1f} mm")
print(f"Duree : {decision['duration_min']} min")
print(f"Raison : {decision['reason']}")
```

Listing 10.2 – Système de décision d'irrigation basé sur l'évapotranspiration

10.4 Drones et Imagerie Agricole

10.4.1 Applications des Drones

TABLE 10.2 – Applications des drones en agriculture

Application	Technologie	Bénéfices
Cartographie parcellaire	RGB, LIDAR	Topographie, drainage
Stress hydrique	Thermique (LWIR)	Détection zones sèches
Santé des plantes	Multispectral (NDVI)	Chlorophylle, vigueur
Comptage plants	Vision + IA	Densité, gaps
Épandage ciblé	GPS RTK	Réduction intrants 30%

10.4.2 Indice de Végétation NDVI

L'indice NDVI (Normalized Difference Vegetation Index) est calculé à partir des bandes rouge (R) et proche infrarouge (NIR) :

$$NDVI = \frac{NIR - R}{NIR + R} \tag{10.1}$$

TABLE 10.3 – Interprétation des valeurs NDVI

NDVI	État	Action
0.8 - 1.0	Végétation très dense et saine	Aucune
0.6 - 0.8	Végétation saine	Surveillance
0.4 - 0.6	Stress modéré	Investigation
0.2 - 0.4	Stress sévère	Intervention urgente
< 0.2	Sol nu ou végétation morte	-

Chapitre 11

Plateformes IoT et Services Cloud

11.1 Introduction aux Plateformes IoT

Une **plateforme IoT** est un ensemble de services logiciels permettant de connecter, gérer et exploiter des dispositifs IoT à grande échelle. Elle assure la connectivité, le stockage des données, l'analyse et l'intégration avec les applications métier.

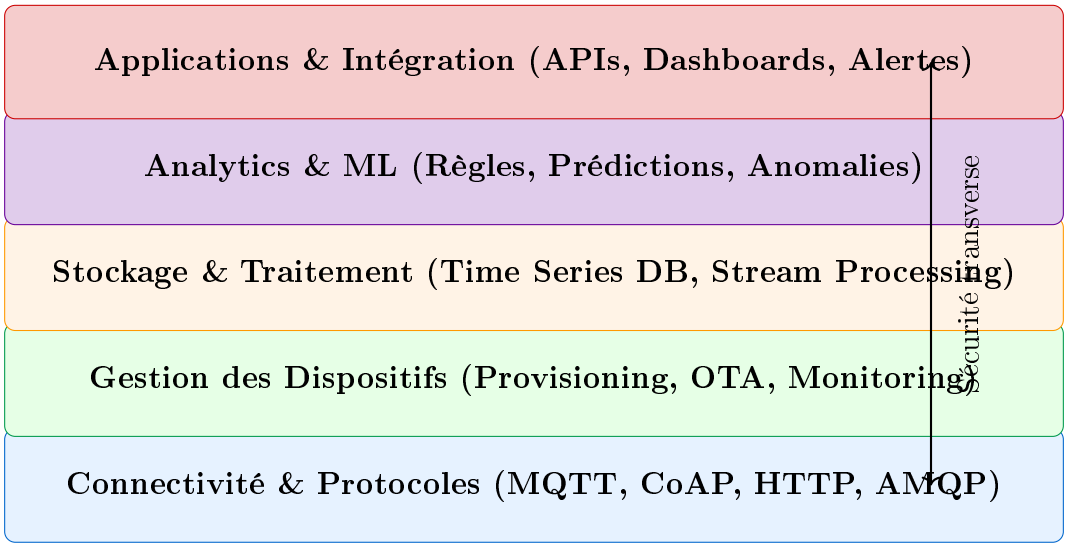


FIGURE 11.1 – Architecture d'une plateforme IoT

11.2 Plateformes Cloud Majeures

11.2.1 AWS IoT

Amazon Web Services propose une suite complète de services IoT :

TABLE 11.1 – Services AWS IoT

Service	Fonction	Cas d'usage
AWS IoT Core	Broker MQTT, règles, shadows	Connectivité
AWS IoT Greengrass	Edge computing, Lambda local	Traitement local
AWS IoT Analytics	Pipeline données, Data Lake	Analyse historique
AWS IoT SiteWise	Données industrielles, OPC UA	IIoT
AWS IoT Events	Détection d'événements, alertes	Monitoring
AWS IoT TwinMaker	Jumeaux numériques 3D	Simulation

11.2.2 Azure IoT

Microsoft Azure propose une approche intégrée :

TABLE 11.2 – Services Azure IoT

Service	Fonction	Cas d'usage
Azure IoT Hub	Ingestion bidirectionnelle	Connectivité centrale
Azure IoT Central	Plateforme SaaS clé en main	Prototypage rapide
Azure IoT Edge	Runtime edge, modules conteneurs	Edge AI
Azure Digital Twins	Graphe de jumeaux, DTDL	Modélisation
Azure Stream Analytics	Traitement temps réel	CEP
Azure Time Series Insights	TSDB, exploration données	Historique

11.2.3 Google Cloud IoT

TABLE 11.3 – Services Google Cloud pour IoT

Service	Fonction	Cas d'usage
Cloud IoT Core	Gestion dispositifs, MQTT/HTTP	Connectivité
Cloud Pub/Sub	Messagerie asynchrone	Découplage
Cloud Dataflow	Traitement stream/batch	ETL temps réel
BigQuery	Data warehouse, ML intégré	Analytics
Vertex AI	AutoML, déploiement modèles	Prédiction

11.2.4 Comparaison des Plateformes Cloud

TABLE 11.4 – Comparaison des plateformes IoT cloud

Critère	AWS IoT	Azure IoT	GCP IoT
Protocoles	MQTT, HTTP, LoRaWAN	MQTT, AMQP, HTTP	MQTT, HTTP
Edge computing	Greengrass	IoT Edge	Edge TPU
Jumeaux numériques	TwinMaker	Digital Twins	Limité
ML embarqué	SageMaker Neo	ML.NET, ONNX	TensorFlow Lite
Sécurité	X.509, SigV4	X.509, TPM	JWT, X.509
Tarification	Par message	Par message	Par message
Points forts	Écosystème, Lambda	Intégration MS	BigQuery, AI

11.3 Plateformes Open Source

11.3.1 ThingsBoard

ThingsBoard est une plateforme IoT open source offrant :

— Gestion multi-tenant des dispositifs
— Tableaux de bord personnalisables (drag-and-drop)
— Moteur de règles visuel
— Support MQTT, CoAP, HTTP, LwM2M
— Intégration Kafka, RabbitMQ

11.3.2 Eclipse IoT

L'écosystème Eclipse IoT comprend :

— **Eclipse Mosquitto** : broker MQTT léger
— **Eclipse Ditto** : jumeaux numériques
— **Eclipse Hono** : connectivité multi-protocoles
— **Eclipse Kapua** : plateforme complète
— **Eclipse hawkBit** : mise à jour OTA

11.3.3 Exemple : Déploiement ThingsBoard

```yaml
# docker-compose.yml
version: '3.0'
services:
  thingsboard:
    image: thingsboard/tb-postgres
    ports:
      - "8080:9090"      # Web UI
      - "1883:1883"      # MQTT
      - "5683:5683/udp" # CoAP
    environment:
      TB_QUEUE_TYPE: in-memory
    volumes:
      - tb-data:/data
      - tb-logs:/var/log/thingsboard
    restart: always

  postgres:
    image: postgres:12
    environment:
      POSTGRES_DB: thingsboard
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
    volumes:
      - postgres-data:/var/lib/postgresql/data
    restart: always
```

```
26
27 volumes:
28   tb-data:
29   tb-logs:
30   postgres-data:
```

Listing 11.1 – Déploiement ThingsBoard avec Docker Compose

11.4 Intégration et APIs

11.4.1 Patterns d'Intégration

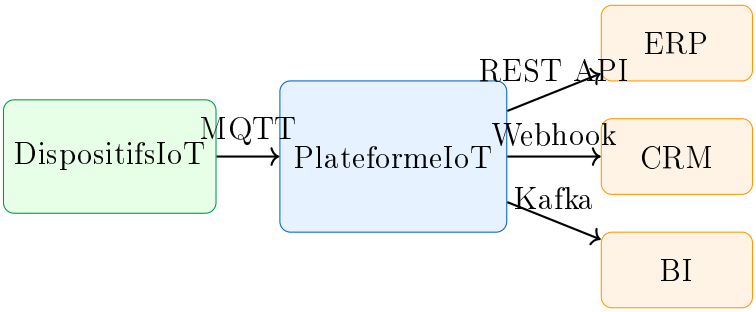


FIGURE 11.2 – Patterns d'intégration IoT

11.4.2 Exemple d'API REST IoT

```python
from fastapi import FastAPI, HTTPException, Depends
from pydantic import BaseModel
from datetime import datetime
from typing import List, Optional
import databases
import sqlalchemy

# Configuration
DATABASE_URL = "postgresql://user:pass@localhost/iot_db"
database = databases.Database(DATABASE_URL)

app = FastAPI(title="IoT Platform API", version="1.0")

# Mod les Pydantic
class DeviceCreate(BaseModel):
    device_id: str
    device_type: str
    location: Optional[str] = None
    metadata: Optional[dict] = {}

class TelemetryData(BaseModel):
    device_id: str
    timestamp: datetime
```

```
    data: dict

class DeviceResponse(BaseModel):
    id: int
    device_id: str
    device_type: str
    status: str
    last_seen: Optional[datetime]

# Endpoints dispositifs
@app.post("/api/v1/devices", response_model=DeviceResponse)
async def create_device(device: DeviceCreate):
    """Enregistrer un nouveau dispositif"""
    query = devices.insert().values(
        device_id=device.device_id,
        device_type=device.device_type,
        location=device.location,
        status="registered"
    )
    device_db_id = await database.execute(query)
    return {**device.dict(), "id": device_db_id, "status": "
       registered"}

@app.get("/api/v1/devices", response_model=List[DeviceResponse])
async def list_devices(
    device_type: Optional[str] = None,
    status: Optional[str] = None,
    limit: int = 100
):
    """Lister les dispositifs avec filtres"""
    query = devices.select().limit(limit)
    if device_type:
        query = query.where(devices.c.device_type == device_type)
    if status:
        query = query.where(devices.c.status == status)
    return await database.fetch_all(query)

@app.get("/api/v1/devices/{device_id}")
async def get_device(device_id: str):
    """Obtenir les d tails d'un dispositif"""
    query = devices.select().where(devices.c.device_id == device_id
       )
    device = await database.fetch_one(query)
    if not device:
        raise HTTPException(status_code=404, detail="Device␣not␣
           found")
    return device

# Endpoints t l m trie
@app.post("/api/v1/telemetry")
async def ingest_telemetry(data: TelemetryData):
```

```
    """Ing rer des donn es de t l m trie"""
    # V rifier que le device existe
    device = await get_device(data.device_id)

    # Stocker en time series
    query = telemetry.insert().values(
        device_id=data.device_id,
        timestamp=data.timestamp,
        data=data.data
    )
    await database.execute(query)

    # Mettre    jour last_seen
    update_query = devices.update().where(
        devices.c.device_id == data.device_id
    ).values(last_seen=data.timestamp)
    await database.execute(update_query)

    return {"status": "ok", "device_id": data.device_id}

@app.get("/api/v1/telemetry/{device_id}")
async def get_telemetry(
    device_id: str,
    start_time: Optional[datetime] = None,
    end_time: Optional[datetime] = None,
    limit: int = 1000
):
    """R cup rer l'historique de t l m trie"""
    query = telemetry.select().where(
        telemetry.c.device_id == device_id
    ).order_by(telemetry.c.timestamp.desc()).limit(limit)

    if start_time:
        query = query.where(telemetry.c.timestamp >= start_time)
    if end_time:
        query = query.where(telemetry.c.timestamp <= end_time)

    return await database.fetch_all(query)

# Endpoints alertes
@app.get("/api/v1/alerts")
async def get_alerts(
    device_id: Optional[str] = None,
    severity: Optional[str] = None,
    acknowledged: bool = False
):
    """R cup rer les alertes actives"""
    query = alerts.select().where(alerts.c.acknowledged ==
       acknowledged)
    if device_id:
        query = query.where(alerts.c.device_id == device_id)
```

```
122     if severity:
123         query = query.where(alerts.c.severity == severity)
124     return await database.fetch_all(query)
125 
126 @app.startup_event
127 async def startup():
128     await database.connect()
129 
130 @app.shutdown_event
131 async def shutdown():
132     await database.disconnect()
```

Listing 11.2 – API REST pour plateforme IoT avec FastAPI

Conclusion

L'Internet des Objets constitue un paradigme technologique transformateur qui redéfinit notre interaction avec le monde physique. Ce cours approfondi a présenté l'ensemble des fondements et applications de l'IoT :

— Les **concepts fondamentaux** : objets connectés, capteurs, actionneurs et architecture des microcontrôleurs (Arduino, ESP32, Raspberry Pi, STM32)

— La **pile protocolaire** complète adaptée aux contraintes des dispositifs connectés, de la couche physique (LoRa, BLE, ZigBee, NB-IoT) aux protocoles applicatifs (MQTT, CoAP, AMQP)

— Les **architectures de déploiement** Edge-Fog-Cloud pour un traitement optimisé des données selon les contraintes de latence et de bande passante

— Les **enjeux de sécurité** critiques et les bonnes pratiques pour protéger les systèmes IoT contre les cybermenaces

— L'**Intelligence Artificielle** et le Machine Learning embarqué (TinyML) pour l'analyse des données et la prise de décision autonome

— Les applications sectorielles majeures : **Industrie 4.0**, **Ville Intelligente**, **Santé Connectée**, **Agriculture de Précision**

— Les **plateformes IoT** cloud (AWS, Azure, GCP) et open source (ThingsBoard, Eclipse IoT)

— Des **projets pratiques** complets illustrant la mise en œuvre concrète des concepts

Perspectives et Tendances Futures

L'IoT continue d'évoluer rapidement, porté par plusieurs tendances majeures :

1. **5G et 6G** : latence ultra-faible ($<$1ms), massive IoT (1M devices/km^2), slicing réseau
2. **TinyML et Edge AI** : intelligence artificielle embarquée sur microcontrôleurs, inférence locale
3. **Sustainability IoT** : optimisation énergétique, économie circulaire, empreinte carbone
4. **Digital Twins 2.0** : jumeaux numériques autonomes, simulation prédictive
5. **Quantum-Safe IoT** : cryptographie post-quantique pour sécuriser les communications
6. **Blockchain IoT** : traçabilité décentralisée, identité souveraine des objets
7. **Human-Machine Teaming** : collaboration homme-robot, interfaces neuronales

La maîtrise des fondamentaux présentés dans ce cours est essentielle pour concevoir des solutions innovantes répondant aux défis technologiques et sociétaux de demain.

Annexe A

Glossaire

ADC	Analog-to-Digital Converter - Convertisseur analogique-numérique
AMQP	Advanced Message Queuing Protocol
API	Application Programming Interface
BLE	Bluetooth Low Energy
CoAP	Constrained Application Protocol
DTLS	Datagram Transport Layer Security
Edge	Traitement à la périphérie du réseau
Fog	Couche intermédiaire entre Edge et Cloud
GPIO	General Purpose Input/Output
I2C	Inter-Integrated Circuit (bus série)
IIoT	Industrial Internet of Things
IoMT	Internet of Medical Things
LoRa	Long Range (modulation radio)
LoRaWAN	LoRa Wide Area Network (protocole)
LPWAN	Low Power Wide Area Network
MCU	Microcontroller Unit
MQTT	Message Queuing Telemetry Transport
NB-IoT	Narrowband IoT (cellulaire)
OPC UA	Open Platform Communications Unified Architecture
OTA	Over-The-Air (mise à jour)
PLC	Programmable Logic Controller
PWM	Pulse Width Modulation
QoS	Quality of Service
REST	Representational State Transfer
RTOS	Real-Time Operating System
RUL	Remaining Useful Life
SCADA	Supervisory Control and Data Acquisition
SPI	Serial Peripheral Interface

TLS	Transport Layer Security
UART	Universal Asynchronous Receiver-Transmitter
VWC	Volumetric Water Content
WSN	Wireless Sensor Network

Annexe B

Références des Composants

B.1 Microcontrôleurs

TABLE B.1 – Références des microcontrôleurs courants

Modèle	Fabricant	Architecture	Documentation
ATmega328P	Microchip	AVR 8-bit	microchip.com/wwwproducts/ATmega328P
ESP32	Espressif	Xtensa LX6	espressif.com/en/products/socs/esp32
STM32F4	STMicroelectronics	ARM Cortex-M4	st.com/en/microcontrollers/stm32f4
nRF52840	Nordic Semi.	ARM Cortex-M4	nordicsemi.com/Products/nRF52840
RP2040	Raspberry Pi	ARM Cortex-M0+	raspberrypi.com/products/rp2040

B.2 Capteurs Courants

TABLE B.2 – Références des capteurs IoT

Capteur	Type	Interface	Tension	Prix
DHT22	Temp + Humid	Digital	3.3-5V	5€
BME280	Temp + Humid + Pres	I2C/SPI	1.8-3.6V	8€
DS18B20	Température	1-Wire	3-5.5V	3€
BH1750	Luminosité	I2C	3-5V	3€
HC-SR04	Ultrasons	Digital	5V	2€
VL53L0X	ToF Laser	I2C	2.6-3.5V	10€
MPU6050	Accéléromètre + Gyro	I2C	3-5V	4€
MQ-2	Gaz (GPL, fumée)	Analog	5V	3€

Bibliographie

Ouvrages Fondamentaux

1. Atzori, L., Iera, A., & Morabito, G. (2010). *The Internet of Things : A survey*. Computer Networks, 54(15), 2787-2805.
2. Gubbi, J., Buyya, R., Marusic, S., & Palaniswami, M. (2013). *Internet of Things (IoT) : A vision, architectural elements, and future directions*. Future Generation Computer Systems, 29(7), 1645-1660.
3. Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). *Internet of Things : A Survey on Enabling Technologies, Protocols, and Applications*. IEEE Communications Surveys & Tutorials, 17(4), 2347-2376.
4. Minerva, R., Biru, A., & Rotondi, D. (2015). *Towards a definition of the Internet of Things (IoT)*. IEEE Internet Initiative, 1(1), 1-86.
5. Sethi, P., & Sarangi, S. R. (2017). *Internet of Things : Architectures, Protocols, and Applications*. Journal of Electrical and Computer Engineering, 2017.

Edge et Fog Computing

6. Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). *Edge Computing : Vision and Challenges*. IEEE Internet of Things Journal, 3(5), 637-646.
7. Bonomi, F., Milito, R., Zhu, J., & Addepalli, S. (2012). *Fog Computing and Its Role in the Internet of Things*. MCC Workshop on Mobile Cloud Computing, 13-16.
8. Satyanarayanan, M. (2017). *The Emergence of Edge Computing*. Computer, 50(1), 30-39.
9. Yi, S., Li, C., & Li, Q. (2015). *A Survey of Fog Computing : Concepts, Applications and Issues*. Workshop on Mobile Big Data, 37-42.

Protocoles et Communications

10. Bormann, C., Ersue, M., & Keränen, A. (2014). *Terminology for Constrained-Node Networks*. RFC 7228, IETF.
11. Shelby, Z., Hartke, K., & Bormann, C. (2014). *The Constrained Application Protocol (CoAP)*. RFC 7252, IETF.
12. OASIS. (2019). *MQTT Version 5.0*. OASIS Standard.
13. Raza, S., Wallgren, L., & Voigt, T. (2013). *SVELTE : Real-time intrusion detection in the Internet of Things*. Ad Hoc Networks, 11(8), 2661-2674.

14. Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., & Watteyne, T. (2017). *Understanding the limits of LoRaWAN.* IEEE Communications Magazine, 55(9), 34-40.

Sécurité IoT

15. Roman, R., Zhou, J., & Lopez, J. (2013). *On the features and challenges of security and privacy in distributed internet of things.* Computer Networks, 57(10), 2266-2279.

16. Sicari, S., Rizzardi, A., Grieco, L. A., & Coen-Porisini, A. (2015). *Security, privacy and trust in Internet of Things : The road ahead.* Computer Networks, 76, 146-164.

17. Kolias, C., Kambourakis, G., Stavrou, A., & Voas, J. (2017). *DDoS in the IoT : Mirai and Other Botnets.* Computer, 50(7), 80-84.

18. Lin, J., Yu, W., Zhang, N., Yang, X., Zhang, H., & Zhao, W. (2017). *A Survey on Internet of Things : Architecture, Enabling Technologies, Security and Privacy, and Applications.* IEEE IoT Journal, 4(5), 1125-1142.

Intelligence Artificielle et Machine Learning

19. Warden, P., & Situnayake, D. (2020). *TinyML : Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers.* O'Reilly Media.

20. Mohammadi, M., Al-Fuqaha, A., Sorour, S., & Guizani, M. (2018). *Deep Learning for IoT Big Data and Streaming Analytics : A Survey.* IEEE Communications Surveys & Tutorials, 20(4), 2923-2960.

21. Li, T., Sahu, A. K., Talwalkar, A., & Smith, V. (2020). *Federated Learning : Challenges, Methods, and Future Directions.* IEEE Signal Processing Magazine, 37(3), 50-60.

22. Mahdavinejad, M. S., Rezvan, M., Barekatain, M., et al. (2018). *Machine learning for Internet of Things data analysis : A survey.* Digital Communications and Networks, 4(3), 161-175.

Applications Sectorielles

23. Xu, L. D., He, W., & Li, S. (2014). *Internet of Things in Industries : A Survey.* IEEE Transactions on Industrial Informatics, 10(4), 2233-2243.

24. Zanella, A., Bui, N., Castellani, A., Vangelista, L., & Zorzi, M. (2014). *Internet of Things for Smart Cities.* IEEE IoT Journal, 1(1), 22-32.

25. Islam, S. R., Kwak, D., Kabir, M. H., Hossain, M., & Kwak, K. S. (2015). *The Internet of Things for Health Care : A Comprehensive Survey.* IEEE Access, 3, 678-708.

26. Wolfert, S., Ge, L., Verdouw, C., & Bogaardt, M. J. (2017). *Big Data in Smart Farming – A review.* Agricultural Systems, 153, 69-80.

27. Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). *Deep learning in agriculture : A survey.* Computers and Electronics in Agriculture, 147, 70-90.

Standards et Normes

28. ITU-T Y.2060 (2012). *Overview of the Internet of Things*. International Telecommunication Union.
29. ISO/IEC 30141 (2018). *Internet of Things (IoT) — Reference architecture*.
30. IEC 62443 (2018). *Security for industrial automation and control systems*.
31. IEEE 802.15.4 (2020). *Low-Rate Wireless Personal Area Networks*.
32. OPC Foundation (2017). *OPC Unified Architecture Specification*.